

Best Practices in SDK Development

Building a Great Developer Experience

Questions

Questions



Daniel Saidi



danielsaidi.com

[@danielsaidi](https://www.instagram.com/danielsaidi)

Daniel Saidi



danielsaidi.com

[@danielsaidi](https://www.instagram.com/danielsaidi)

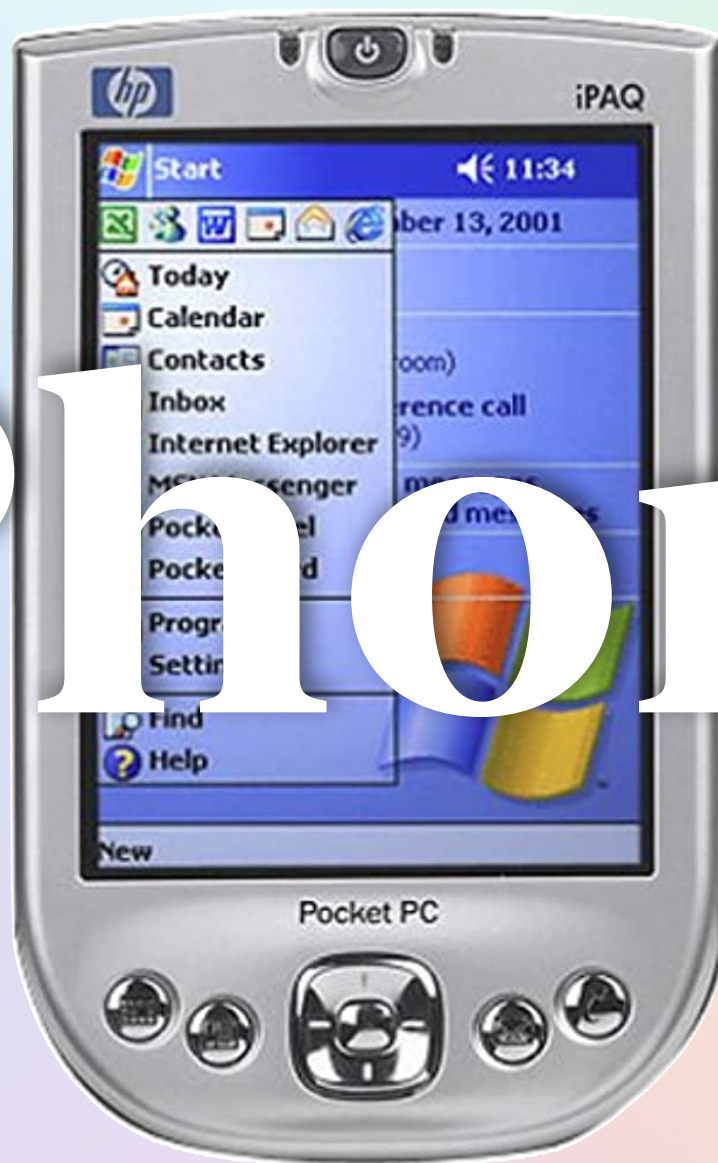


Internet

Mobile



iPhone





SDKs

SDK

SDK?

SDK?

Software Development Kit

What is it?

A complete set of **tools** to help you **build software** for a platform or system.

Consists of

Libraries
Documentation
Resources
etc.

Example

The **iOS SDK** has libraries like UIKit, APIs, and tools like Xcode and Simulator.



A man with glasses and a mustache, wearing a light-colored dress shirt, a patterned tie, and a dark vest, is standing in an office cubicle. He is holding a white mug with both hands and drinking from it. The mug has the word "TECH" visible on it. He is leaning against a wire mesh storage bin on his left. The background shows office cubicles and a fire alarm pull station on the wall.

Safe for Work!

SDKs

SDKs



SDKs



SDKs

SDK

SDK Best Practices

SDK Best Practices

Delight

An SDK should be **delightful** to use, with a public API that **guides developers**.

Trust

An SDK should do what it **promises** and be **stable** over time and versions.

Continuity

An SDK should have a **stable architecture** that evolves with clear **migration paths**.

SDK Best Practices

API Design

System Design

Testing

Documentation

Versioning

Releasing

Developer Tools

Communication

Monetization

API Design

API Design

Guide Your Users

Access Control

API Surface Area

Namespaces

Access Control

```
class MyGreatSDKFeature
```

✗ Types are internal by default, and not part of the public API!

```
public class MyGreatSDKFeature
```

```
class MyGreatSDKFeature
```

```
open class MyGreatSDKFeature
```

✓ Use access modifiers to design your intended public API.


```
public class MyGreatSDKFeature {  
    init() {  
        meaningOfLife = 42  
    }  
  
    var meaningOfLife: Int  
}  
  
extension MyGreatSDKFeature {  
    func calculateTheMeaningOfLife() { meaningOfLife }  
}
```

✗ Type members are also internal by default!

```
public class MyGreatSDKFeature {  
    public init() {  
        meaningOfLife = 42  
    }  
  
    private var meaningOfLife: Int  
}  
  
public extension MyGreatSDKFeature {  
    func calculateTheMeaningOfLife() { meaningOfLife }  
}
```

✓ Pick an intended access modifier for each member/extension.

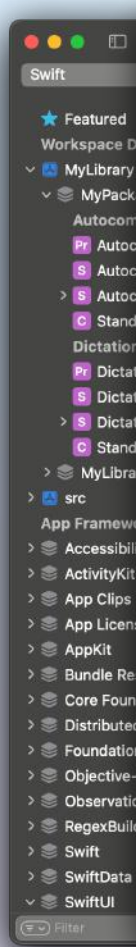
Avoid over-exposing SDK members!

It makes an SDK hard to maintain over time.

API Surface Area

API Surface Area

API Surface Area - Small Project



Swift

- ★ Featured
- Workspace D
- MyLibrary
 - MyPacka
 - Autocom
 - Pr Autoci
 - S Autoci
 - S Autoci
 - C Standi
 - Dictation
 - Pr Dictati
 - S Dictati
 - S Dictati
 - C Standi
 - MyLibrar
- src
- App Frameworks
 - Accessibili
 - ActivityKit
 - App Clips
 - App Licens
 - AppKit
 - Bundle Res
 - Core Foun
 - Distributed
 - Foundatio
 - Objective-C
 - Observatio
 - RegexBuild
 - Swift
 - SwiftData
 - SwiftUI

Filter

Topics

Autocomplete

`protocol` `AutocompleteService`

This protocol can be implemented by types that can perform autocomplete.

`struct` `AutocompleteSuggestion`

This type represents an autocomplete suggestion.

`struct` `AutocompleteToolbar`

This view can be used to show autocomplete toolbars.

`class` `StandardAutocompleteService`

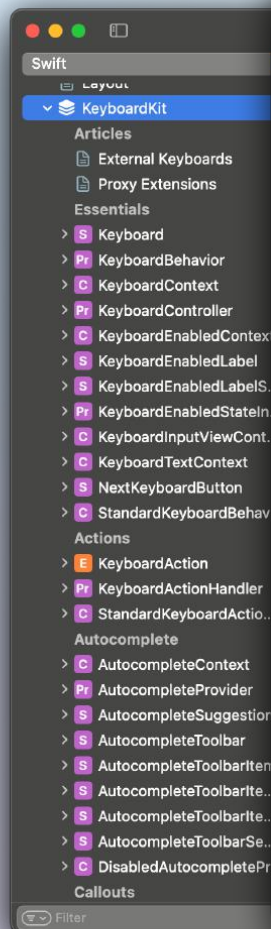
This is a standard autocomplete service implementation.

Dictation



API Surface Area

API Surface Area - Large Project



Autocomplete

class `AutocompleteContext`

This is an observable class that can be used to store a set of autocomplete suggestions.

protocol `AutocompleteProvider`

This protocol can be implemented by any classes that can be used to give autocomplete suggestions as the user types.

struct `AutocompleteSuggestion`

This struct is a standard suggestion that can be used by an autocomplete suggestion provider.

struct `AutocompleteToolbar`

This toolbar can be used to present autocomplete items with separator lines between buttons.

struct `AutocompleteToolbarItem`

This view mimics the autocomplete toolbar item that is used in native iOS keyboards.

struct `AutocompleteToolbarItemSubtitle`

This view mimics the subtitle of autocomplete toolbar items that are used in native iOS keyboards.

struct `AutocompleteToolbarItemTitle`

This view mimics the title of the autocomplete toolbar item that is used in native iOS



Avoid a bloated API surface!

It makes a large SDK hard to navigate.

Namespaces

Namespaces

Namespaces - Options

Structs

Enums

Libraries

Packages

```
public class AutocompleteContext: ObservableObject
public enum AutocompleteEnabledMode
public struct AutocompleteSettings: Codable
public protocol AutocompleteService
public struct AutocompleteSuggestion: Codable
public enum AutocompleteSuggestionType
public struct AutocompleteToolbar: View
public struct AutocompleteToolbarItem: View
public struct AutocompleteToolbarItemStyle: Codable
public struct AutocompleteToolbarSeparator: View
public struct AutocompleteToolbarStyle: Codable
```

✗ Too many top level types will result in a bloated API surface


```
public struct Autocomplete { ... }
```

```
public struct Autocomplete { ... }
```

```
public extension Autocomplete {
```

```
    public class AutocompleteContext: ObservableObject
```

```
    public enum AutocompleteEnabledMode
```

```
    public struct AutocompleteSettings: Codable
```

```
    public protocol AutocompleteService
```

```
    public struct AutocompleteSuggestion: Codable
```

```
    public enum AutocompleteSuggestionType
```

```
    public struct AutocompleteToolbar: View
```

```
    public struct AutocompleteToolbarItem: View
```

```
    public struct AutocompleteToolbarStyle: Codable
```

```
}
```

```
public struct Autocomplete { ... }
```

```
public extension Autocomplete {
```

```
    class AutocompleteContext: ObservableObject
```

```
    enum AutocompleteEnabledMode
```

```
    struct AutocompleteSettings: Codable
```

```
    protocol AutocompleteService
```

```
    struct AutocompleteSuggestion: Codable
```

```
    enum AutocompleteSuggestionType
```

```
    struct AutocompleteToolbar: View
```

```
    struct AutocompleteToolbarItem: View
```

```
    struct AutocompleteToolbarStyle: Codable
```

```
}
```

```
public struct Autocomplete { ... }
```

```
public extension Autocomplete {
```

```
    class Context: ObservableObject
```

```
    enum EnabledMode
```

```
    struct Settings: Codable
```

```
    protocol Service
```

```
    struct Suggestion: Codable
```

```
    enum SuggestionType
```

```
    struct Toolbar: View
```

```
    struct ToolbarItem: View
```

```
    struct ToolbarStyle: Codable
```

```
}
```

```
public struct Autocomplete { ... }

public extension Autocomplete {

    class AutocompleteContext: ObservableObject
    enum AutocompleteEnabledMode
    struct AutocompleteSettings: Codable
    protocol AutocompleteService
    struct AutocompleteSuggestion: Codable
    enum AutocompleteSuggestionType
    struct AutocompleteToolbar: View
    struct AutocompleteToolbarItem: View
    struct AutocompleteToolbarStyle: Codable
}
```

⚠ Too much nesting may not guide the user in the best way!

```
public struct Autocomplete { ... }
```

```
...
```

```
public struct Autocomplete { ... }
```

```
public class AutocompleteContext: ObservableObject
```

```
public struct AutocompleteSettings: Codable
```

```
public protocol AutocompleteService: AnyObject
```

```
...
```

```
public struct Autocomplete { ... }
```

```
public class AutocompleteContext: ObservableObject
```

```
public struct AutocompleteSettings: Codable
```

```
public protocol AutocompleteService: AnyObject
```

```
public extension Autocomplete {
```

```
    enum EnabledMode
```

```
    struct Suggestion: Codable
```

```
    enum SuggestionType
```

```
    struct Toolbar: View
```

```
    struct ToolbarItem: View
```

```
    struct ToolbarStyle: Codable
```

```
}
```

```
...
```


Use well-designed namespaces!

Guide users to the most important parts.

System Design

System Design

Stable

Customizable

Types Over Values

Enums & Structs

Types Over Values

```
public protocol AutocompleteService {  
    var isAutocompleteEnabled: Bool { get set }  
  
    func autocomplete(  
        text: String  
    ) -> [String]  
}
```

```
public protocol AutocompleteService {  
    var isAutocompleteEnabled: Bool { get set }  
    var isAutocorrectEnabled: Bool { get set }  
  
    func autocomplete(  
        text: String  
    ) -> [String]  
}
```

```
public protocol AutocompleteService {  
    var isAutocompleteEnabled: Bool { get set }  
    var isAutocorrectEnabled: Bool { get set }  
  
    func autocomplete(  
        text: String  
    ) -> [Autocomplete.Suggestion]  
}
```



Avoid designing with primitive values!
They couple your API to today's implementation.


```
public protocol AutocompleteService {  
    var isAutocompleteEnabled: Bool { get set }  
    var isAutocorrectEnabled: Bool { get set }  
  
    func autocomplete(  
        text: String  
    ) -> [Autocomplete.Suggestion]  
}
```

```
public protocol AutocompleteService {  
  
    func autocomplete(  
        text: String  
    ) -> [Autocomplete.Suggestion]  
}
```

```
public protocol AutocompleteService {  
    var settings: Autocomplete.Settings { get set }  
  
    func autocomplete(  
        text: String  
    ) -> [Autocomplete.Suggestion]  
}
```

```
public protocol AutocompleteService {  
    func autocomplete(  
        text: String  
    ) -> [Autocomplete.Suggestion]  
}
```

```
public protocol AutocompleteService {  
    func autocomplete(  
        request: Autocomplete.ServiceRequest  
    ) -> [Autocomplete.Suggestion]  
}
```

```
public protocol AutocompleteService {  
    func autocomplete(  
        request: Autocomplete.ServiceRequest  
    ) -> Autocomplete.ServiceResult  
}
```

```
public protocol AutocompleteService {  
    func autocomplete(  
        request: Autocomplete.ServiceRequest  
    ) async throws -> Autocomplete.ServiceResult  
}
```



Prefer to design with types!

It makes your API less likely to break in the future.

Enums & Structs

Enums

Enums

Represent a
closed set of options

Provide exhaustive checks and
compile-time safety

Let the SDK handle
all known cases

Let developers pick from
a list of well-defined options

```
public extension Autocomplete {  
    enum SuggestionType {  
        case autocomplete, autocorrect, unknown  
    }  
}  
...
```

```
public extension Autocomplete {  
    enum SuggestionType {  
        case autocomplete, autocorrect, unknown  
    }  
}  
  
public extension Autocomplete.SuggestionType {  
    var shouldBeAppliedWithWordDelimiter: Bool {  
        switch self {  
            case .autocorrect: true  
            case .autocomplete, .unknown: false  
        }  
    }  
}  
...
```

```
public extension Autocomplete {  
    enum SuggestionType {  
        case autocomplete, autocorrect, unknown  
    }  
}  
...
```

```
public extension Autocomplete {  
    enum SuggestionType {  
        case autocomplete, autocorrect, unknown  
    }  
}  
  
public extension Autocomplete.Toolbar {  
    func backgroundColor(  
        for suggestionType: Autocomplete.SuggestionType  
    ) -> Color {  
        switch suggestionType {  
            case .autocorrect: .white.opacity(0.3)  
            case .autocomplete, .unknown: .clear  
        }  
    }  
}  
...
```

Use enums for a closed set of options!
They provide compile-time safety and guidance.

Structs

Structs

Represent **open-ended**
data structures

Provide controlled
flexibility & customization

Let the SDK provide
standard values

Let developers provide
domain-specific values

```
public extension Autocomplete {  
    struct ToolbarStyle {  
        public init(...) { ... }  
  
        public let textColor: Color  
        public let autocompleteBackgroundColor: Color  
        ...  
    }  
}  
...
```

```
public extension Autocomplete {  
    struct ToolbarStyle {  
        public init(...) { ... }  
  
        public let textColor: Color  
        public let autocompleteBackgroundColor: Color  
        ...  
    }  
}  
  
public extension Autocomplete.ToolbarStyle {  
    static var standard: Self { ... }  
}  
...
```

```
extension Autocomplete.ToolbarStyle {  
  
    static func appStandard(  
        for: AppSpecificState  
    ) -> Self { ... }  
  
    static func disabled(  
        if condition: Bool  
    ) -> Self { ... }  
  
    static func highlighted(  
        if condition: Bool  
    ) -> Self { ... }  
  
    static var standard(  
        for accessibility: AccessibilitySettings  
    ) -> Self { ... }  
}
```

Use structs for *customizable* values!

They allow for flexibility & customization.

Testing

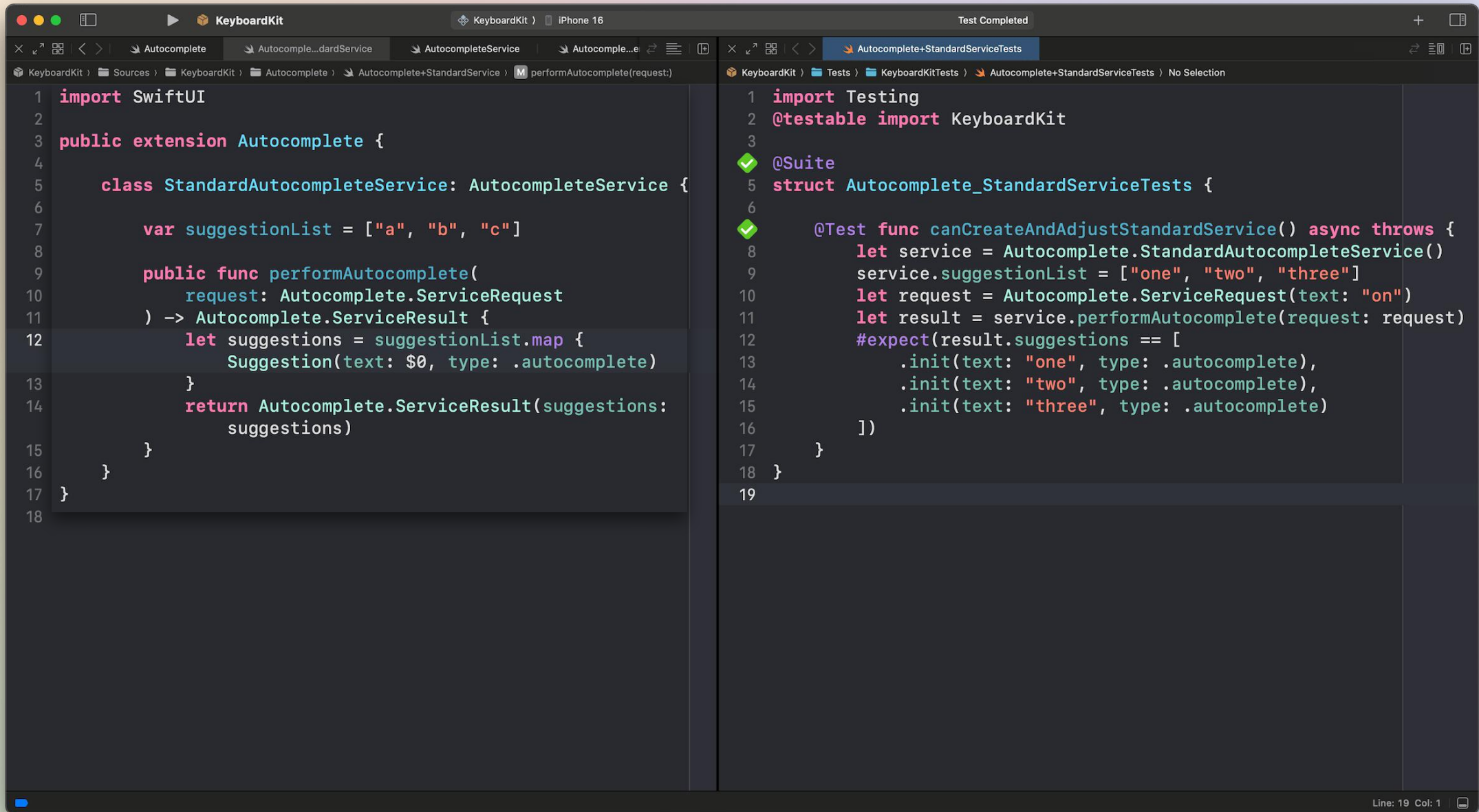
Testing

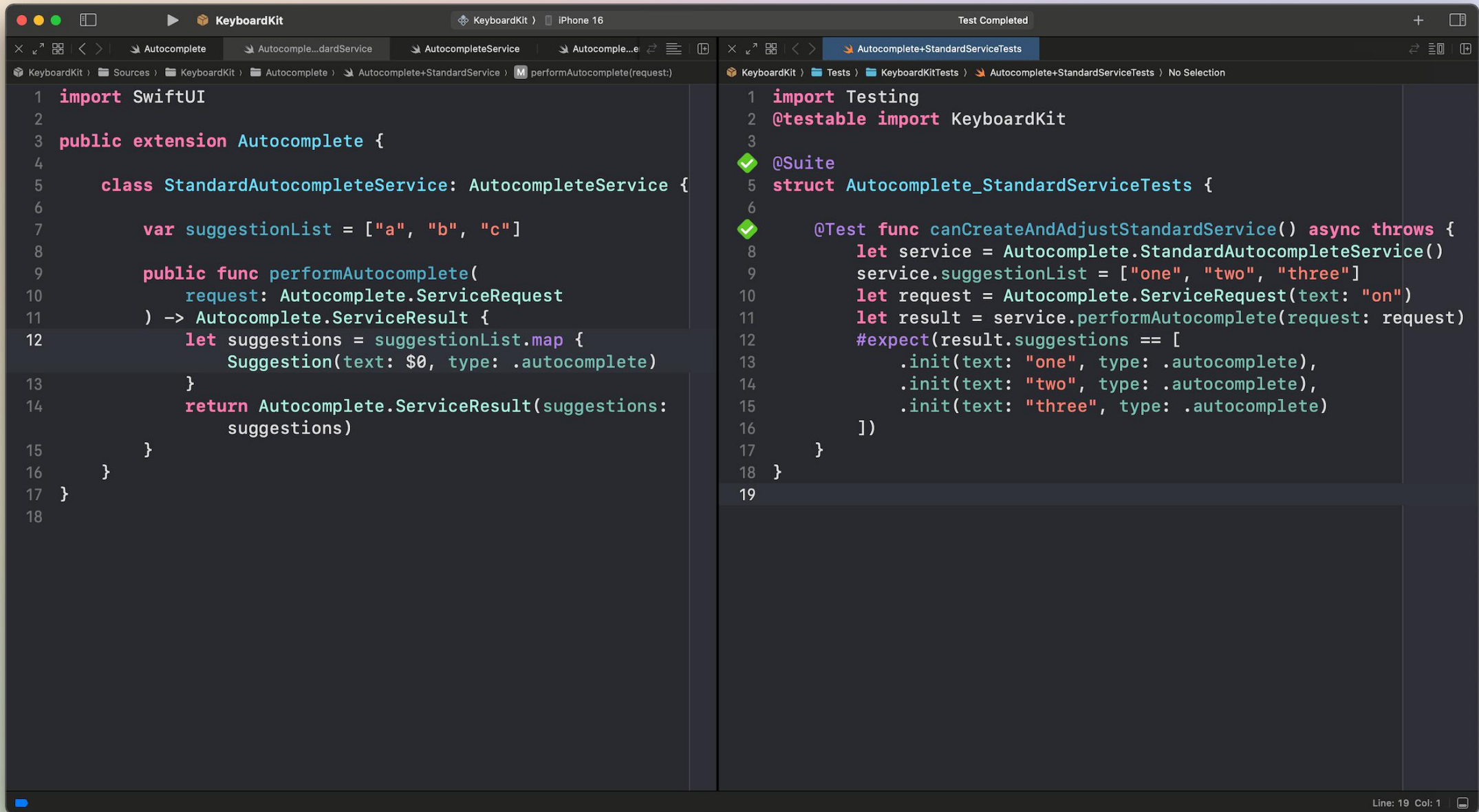
Unit Tests

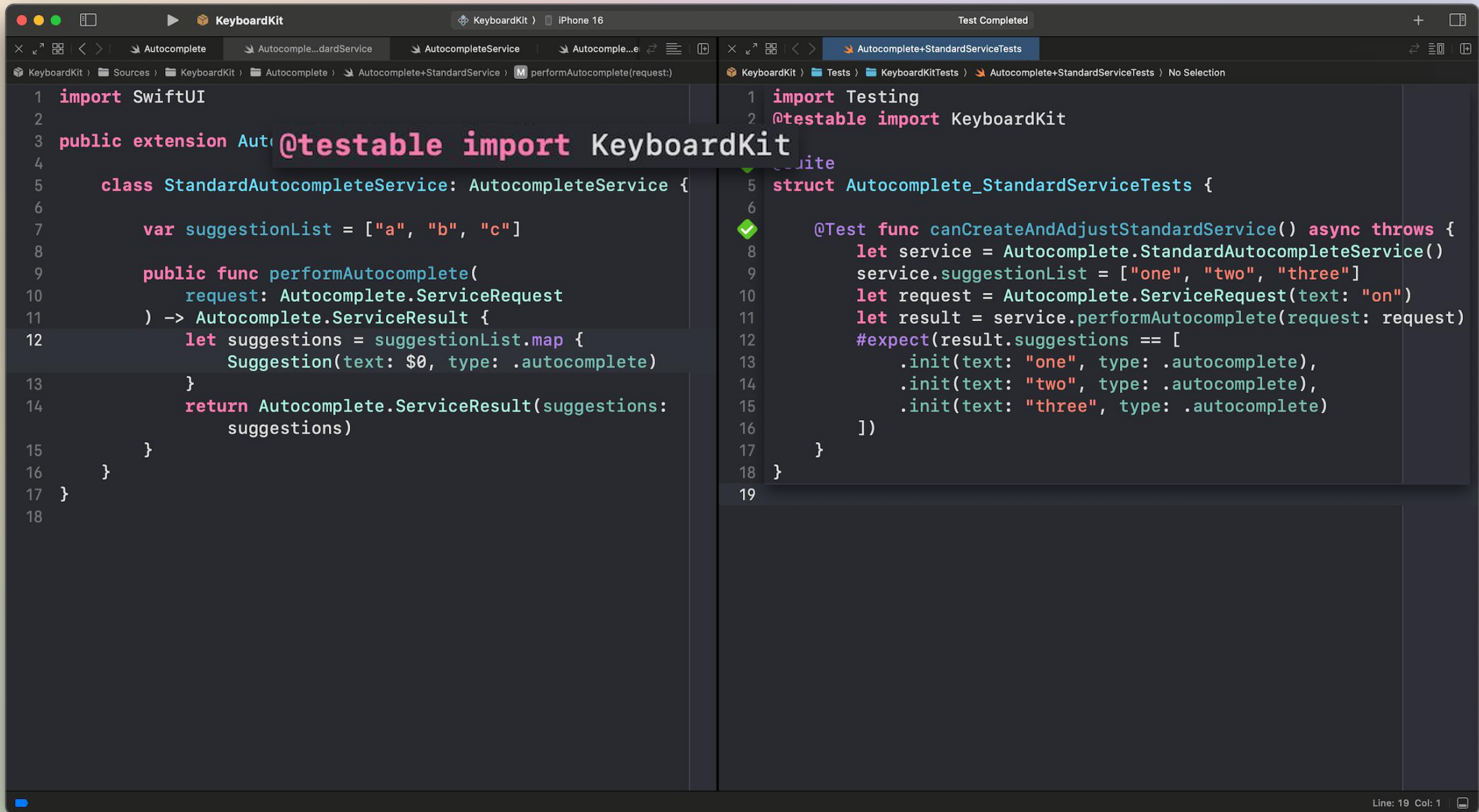
Integration Tests

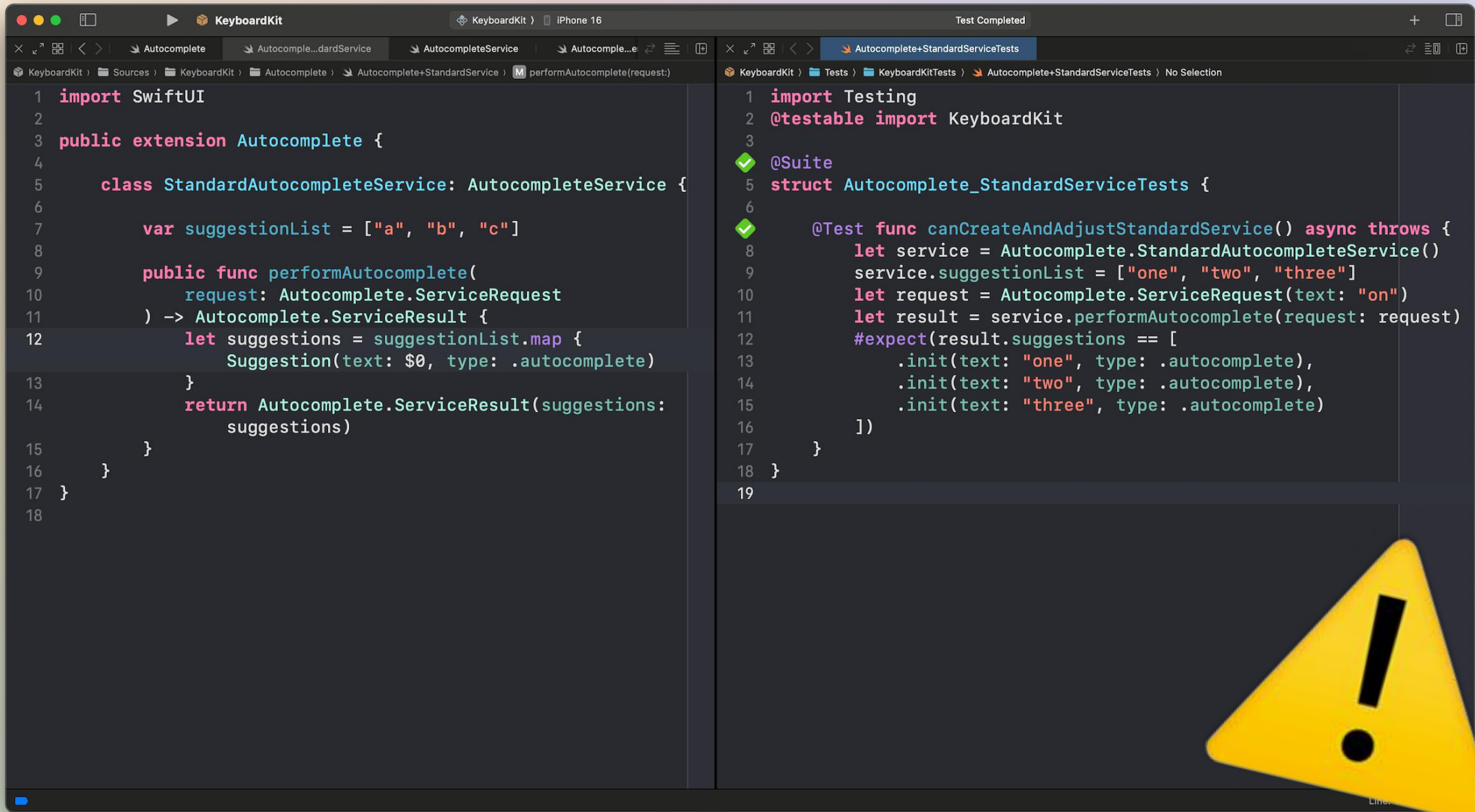
Regression Tests

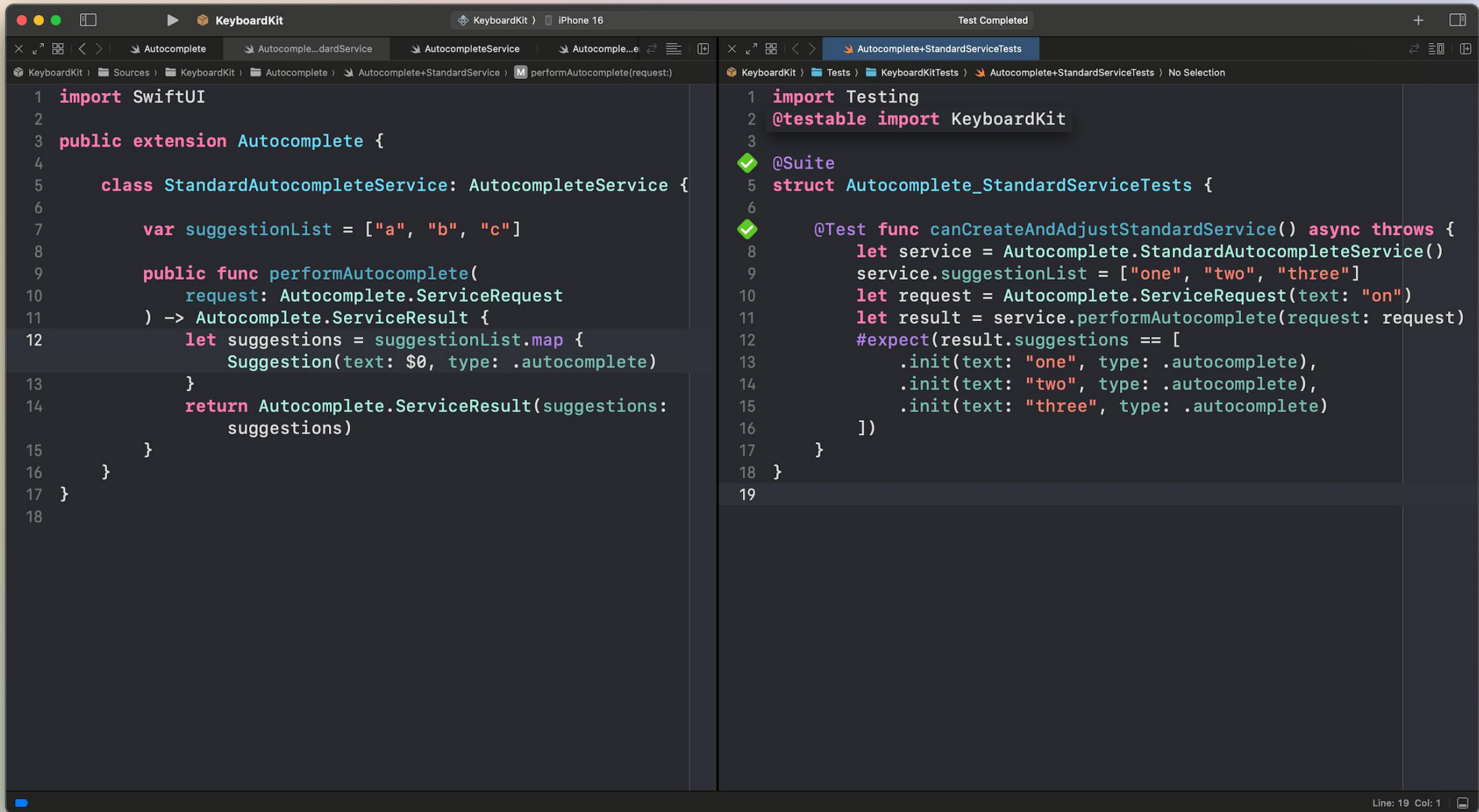
API Design Tests

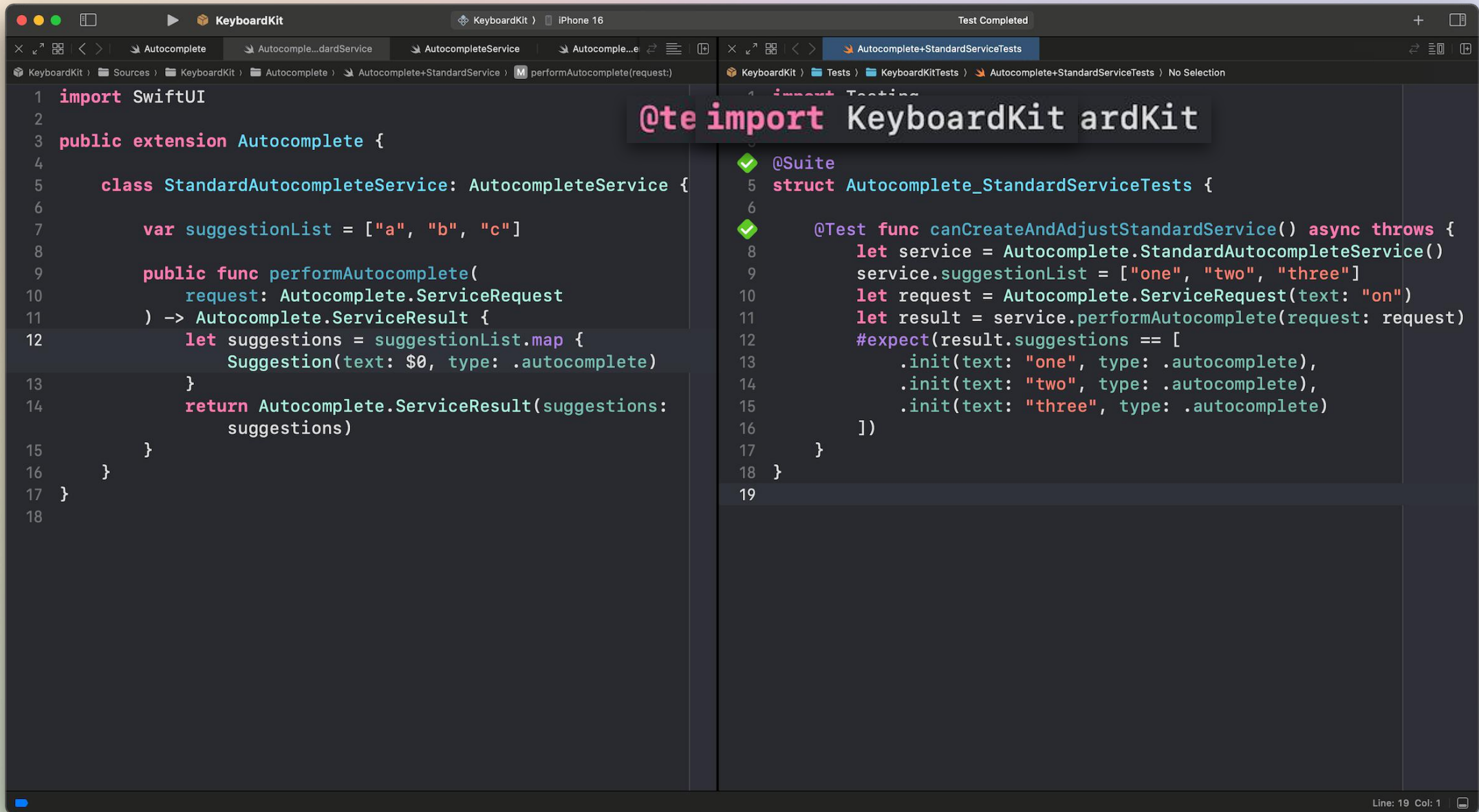


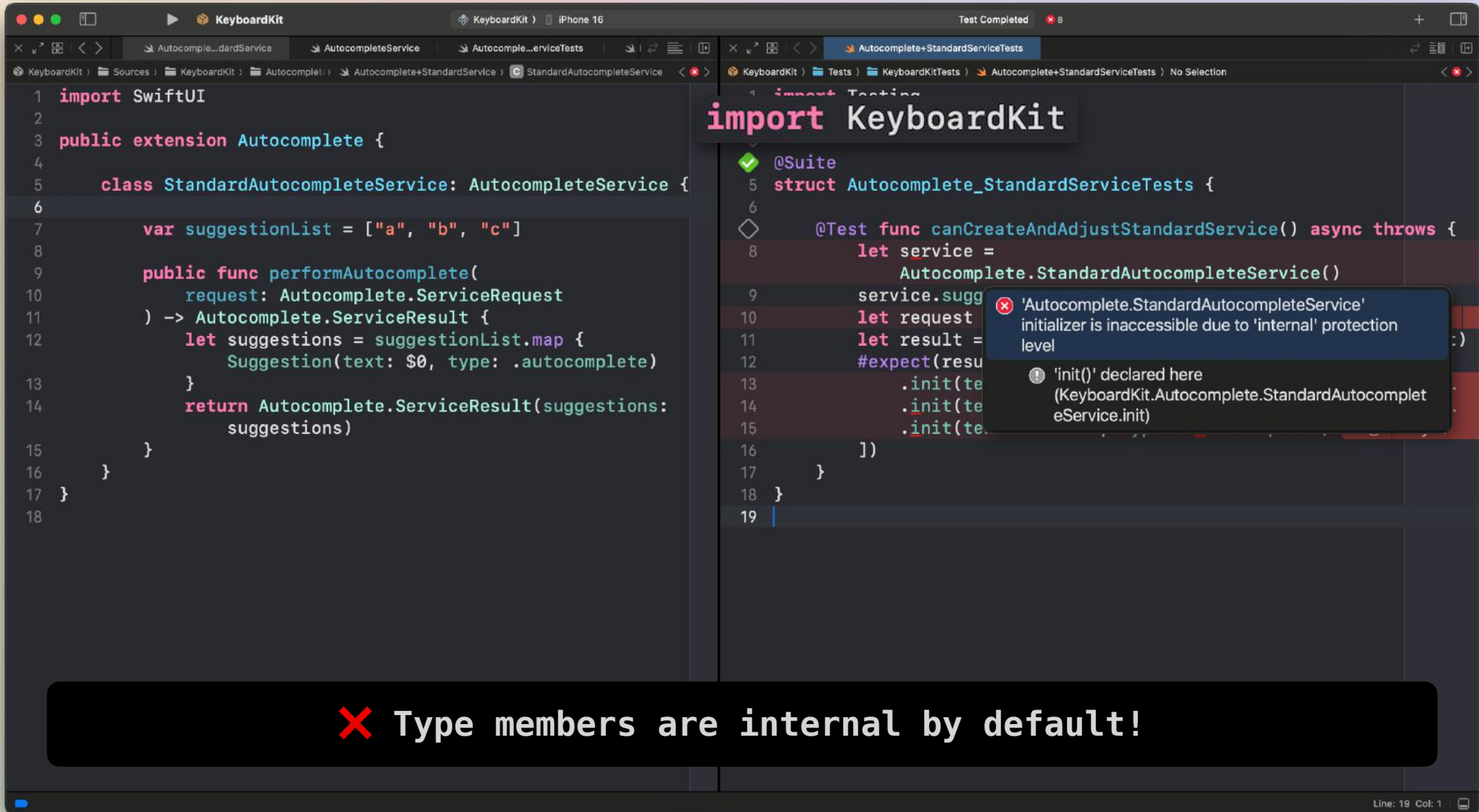


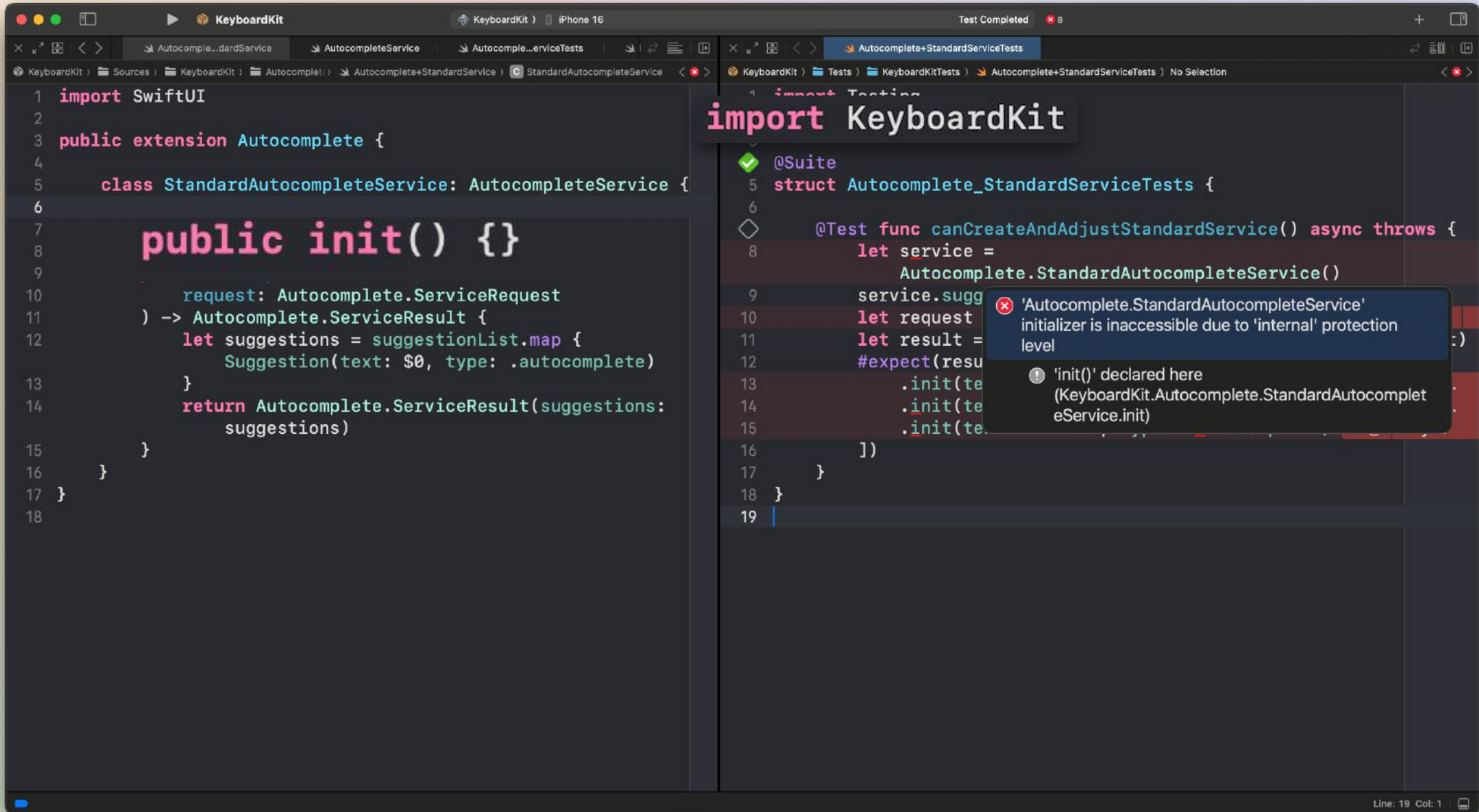


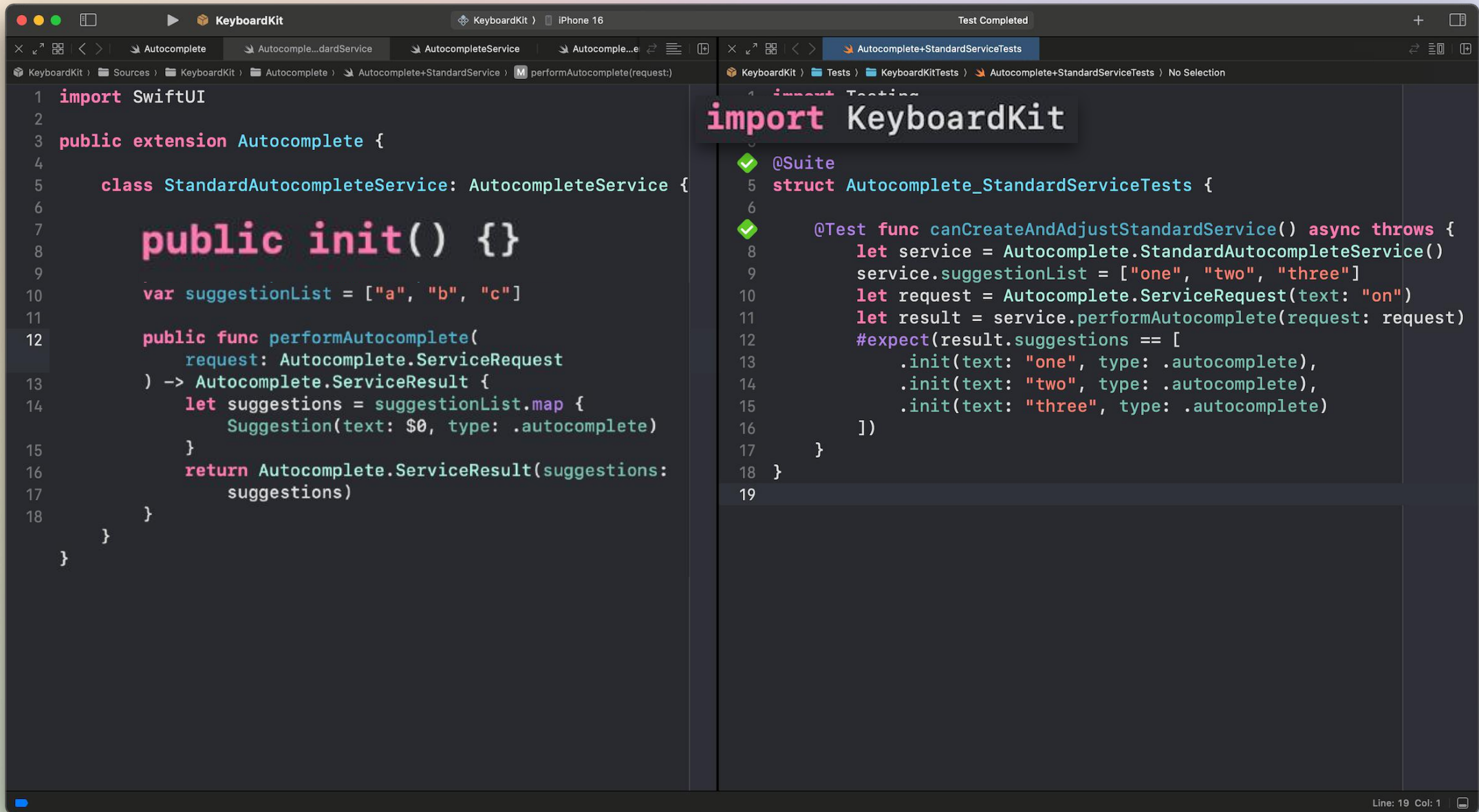












Test your API design!

Separate testable imports from non-testable ones.

Documentation

Documentation

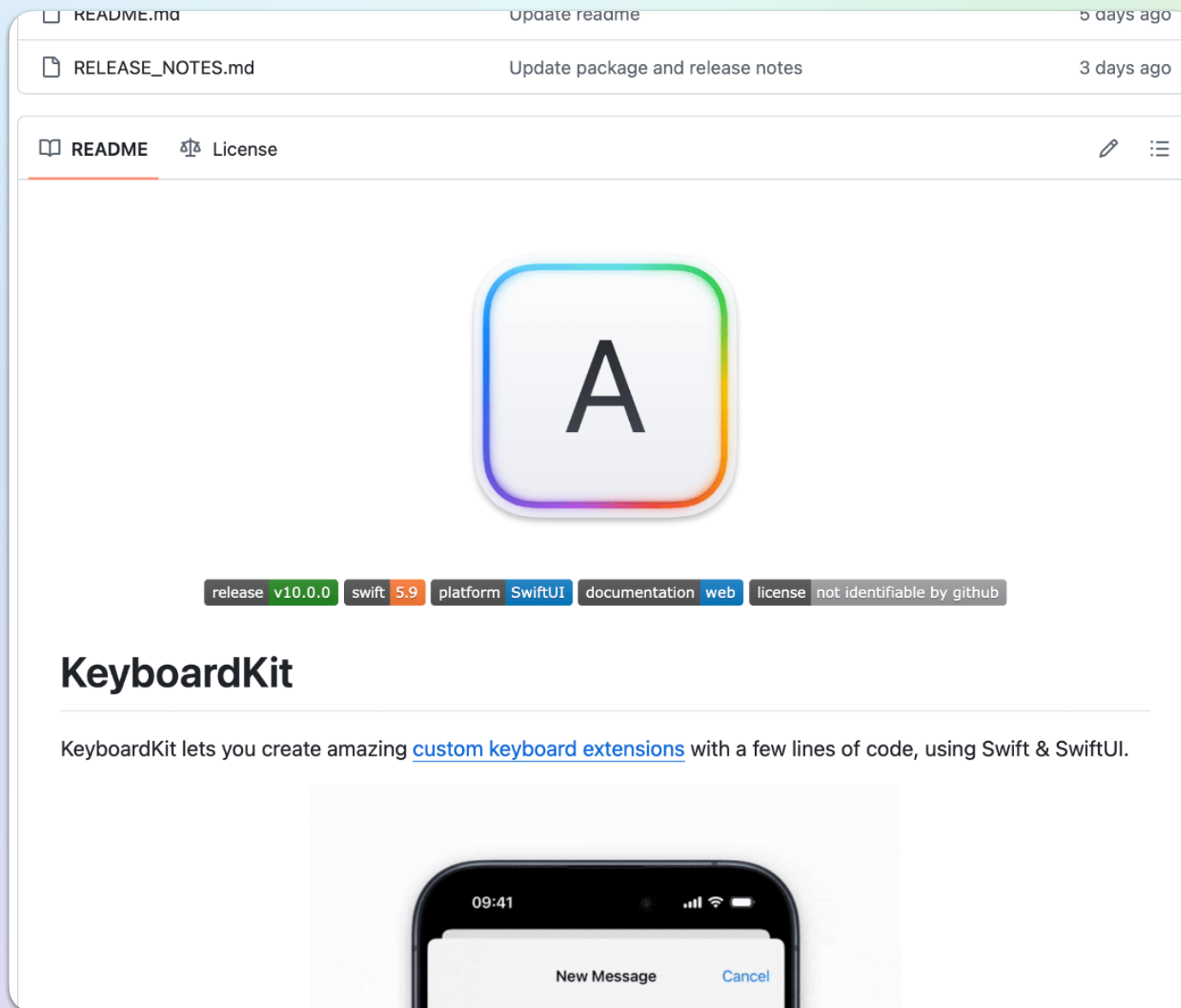
Readme

GitHub Pages

DocC

Code, Articles, Topics, etc.

Readme



README.md

RELEASE_NOTES.md

README License

release v10.0

KeyboardKit

KeyboardKit lets you create

README License

Installation

KeyboardKit can be installed

```
https://github.com/Key
```

KeyboardKit must then be li

KeyboardKit 10

The KeyboardKit 10 is being
demo app, etc.

Use the 10.0.0-dp.1 pre-
important information, dow

Getting Started

The easiest way to set up K

```
import KeyboardKit

extension KeyboardApp

static var keyboa
.init(
name: "KeyboardKit",
```

README License 5 days ago

Features

KeyboardKit provides a free, open-source keyboard engine. [KeyboardKit Pro](#) unlocks more powerful pro features.

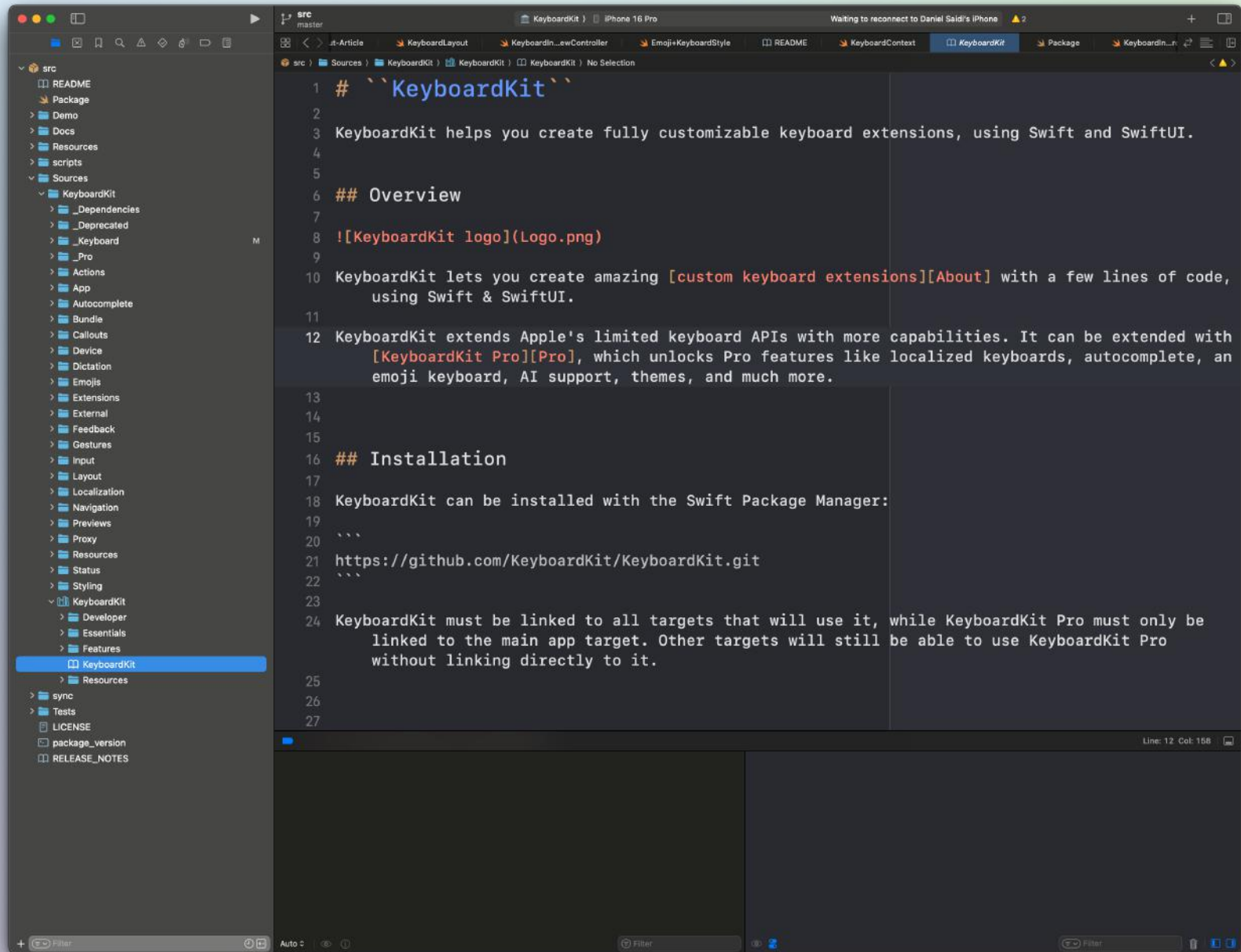
- 🌱 [Essentials](#) - Essential models, services, utilities & views.
- 📱 [Essentials-KeyboardView](#) - A native-looking, customizable keyboard.
- 💥 [Actions](#) - Trigger & handle keyboard-related actions.
- 📱 [App](#) - Set up your app, keyboard, sync settings, etc.
- 🗨️ [Callouts](#) - Show input & secondary action callouts.
- 📱 [Device](#) - Identify device type, capabilities, etc.
- 😊 [Emojis](#) - Emojis, categories, versions, skin tones, etc.
- 🔊 [Feedback](#) - Trigger audio & haptic feedback.
- 👉 [Gestures](#) - Handle a rich set of gestures on any key.
- 🗺️ [Layout](#) - Define and customize dynamic keyboard layouts.
- 🌐 [Localization](#) - Additional locale-related utilities.
- 🗺️ [Navigation](#) - Open urls and other apps from the keyboard.
- 👁️ [Previews](#) - Extensive SwiftUI preview support.
- 📄 [Proxy](#) - Extend the text document proxy with more capabilities.
- ⚙️ [Settings](#) - Provide keyboard settings & link to System Settings.
- 🩺 [Status](#) - Detect if a keyboard is enabled, has full access, etc.
- 🎨 [Styling](#) - Style your keyboard to great extent.

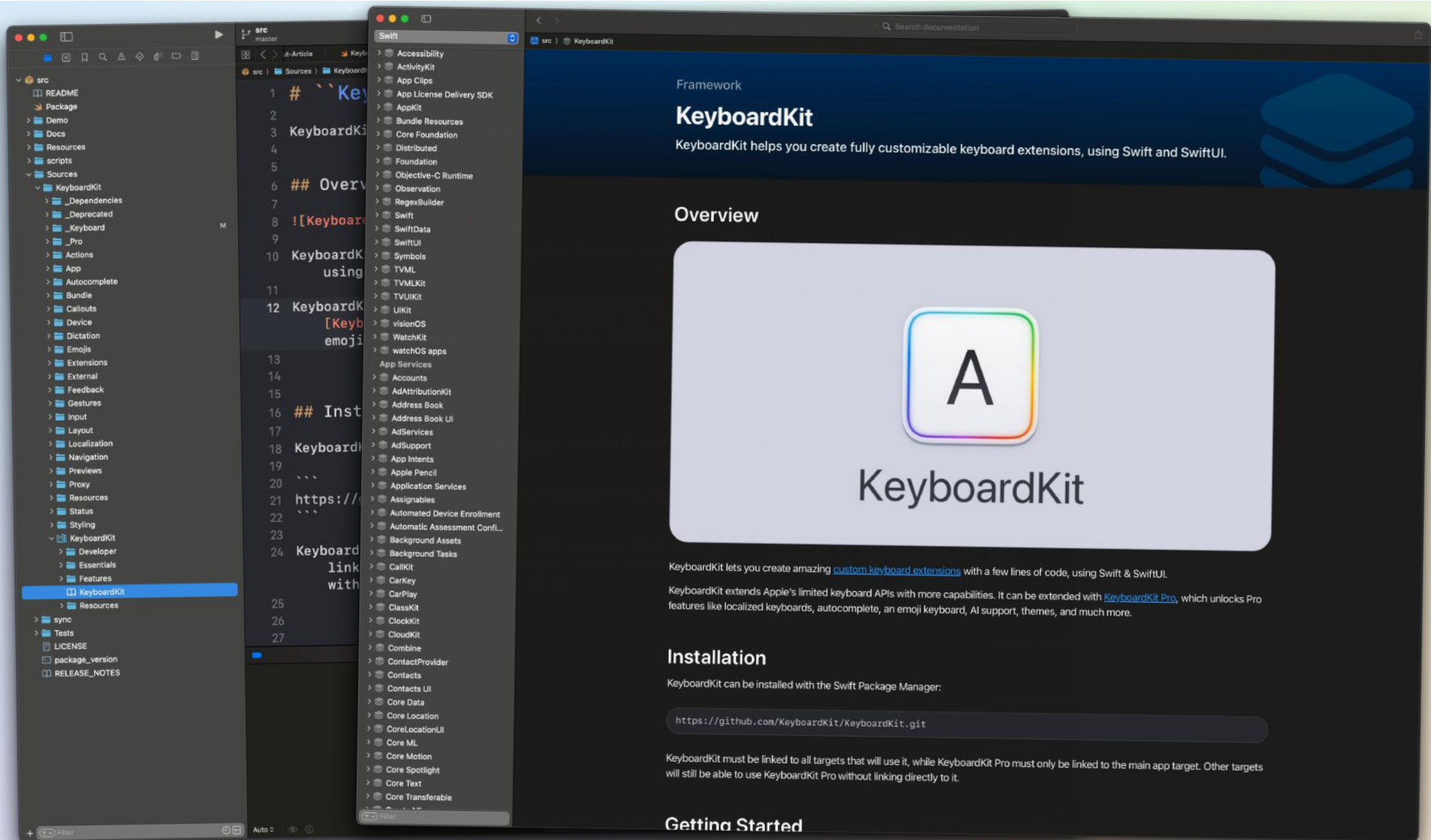
Documentation

The [online documentation](#) has a thorough getting-started guide, a detailed article for each feature, code samples, etc. You can also build it from the source code to get better formatting.

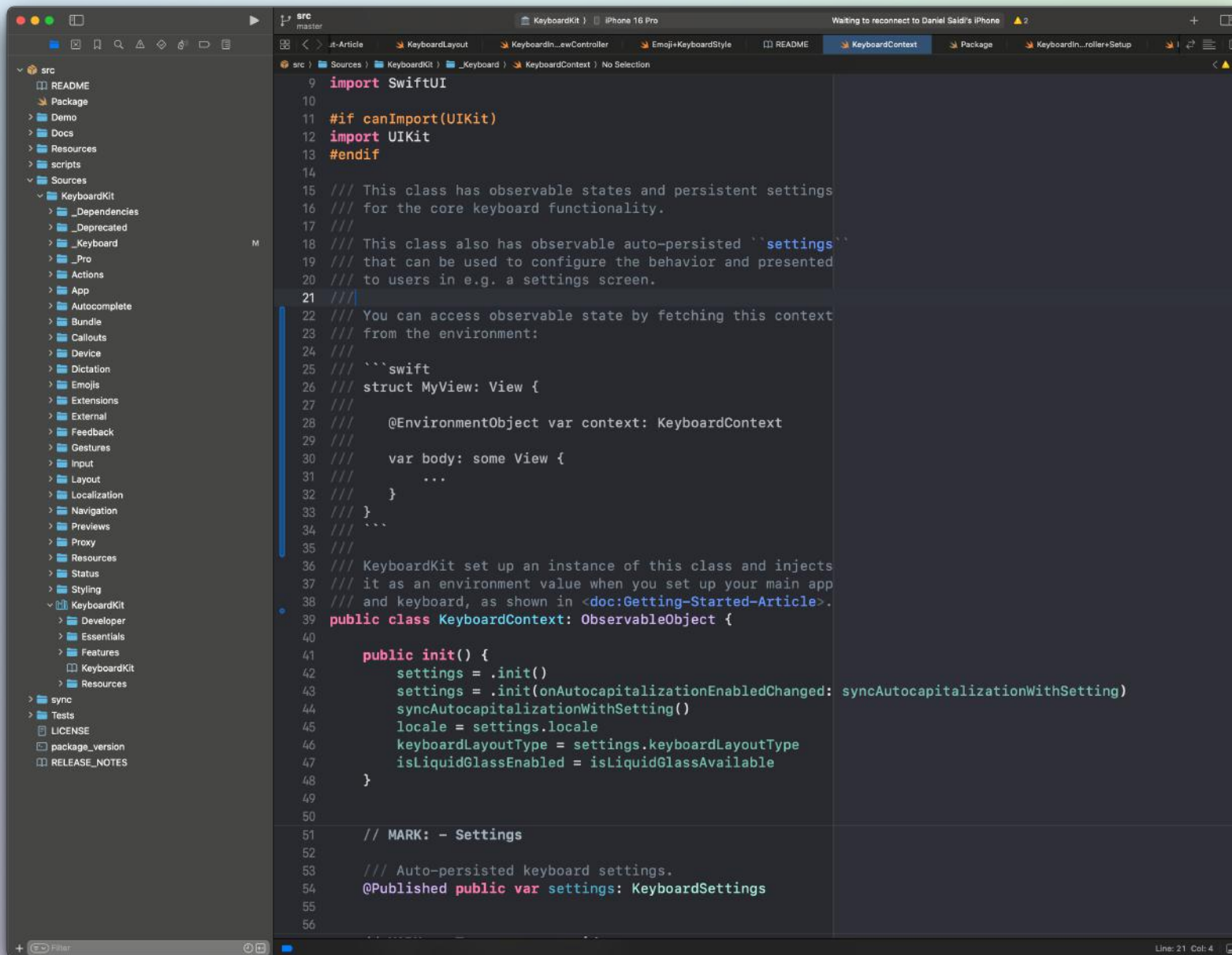
DocC

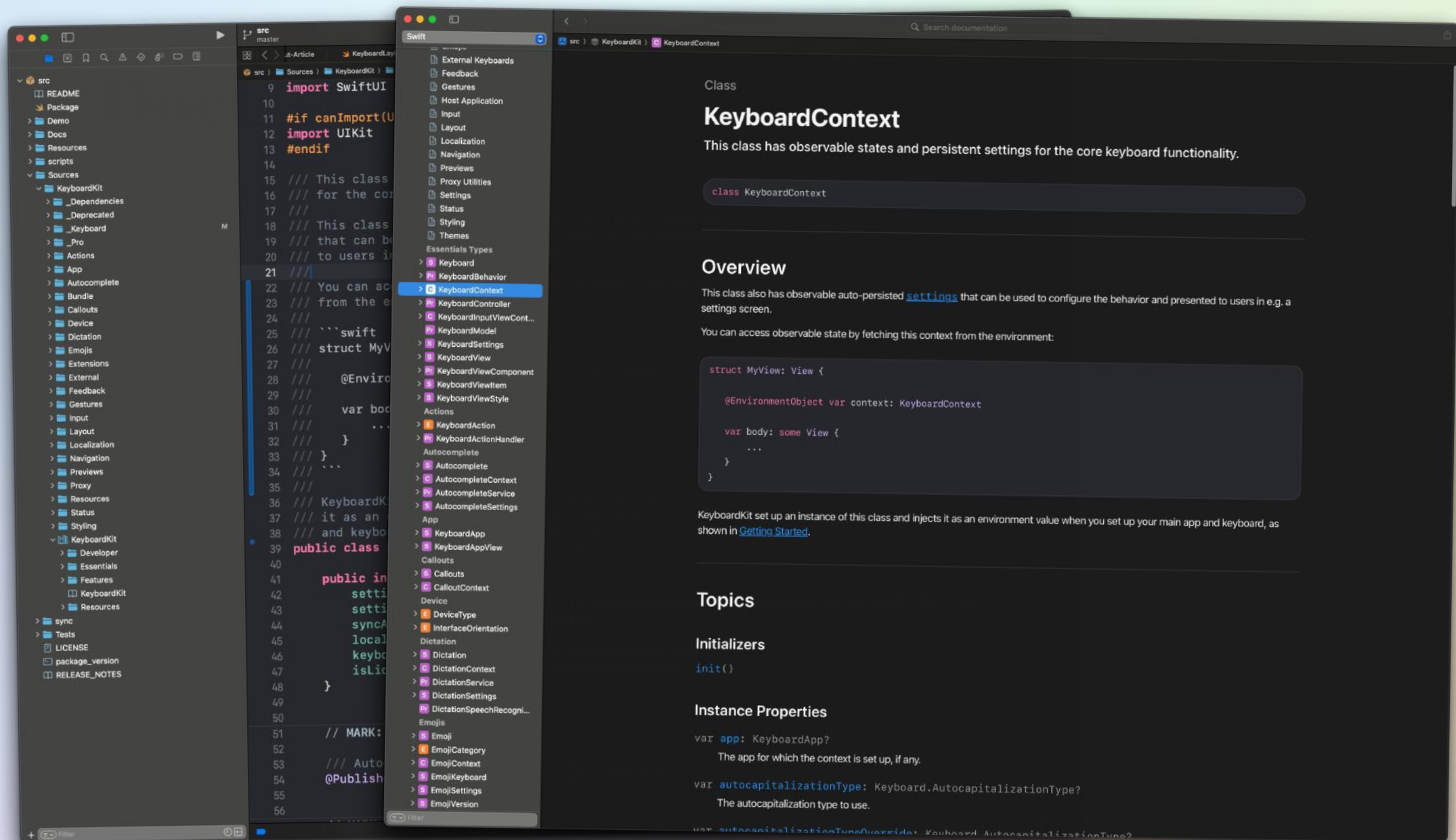
DocC - Main Article



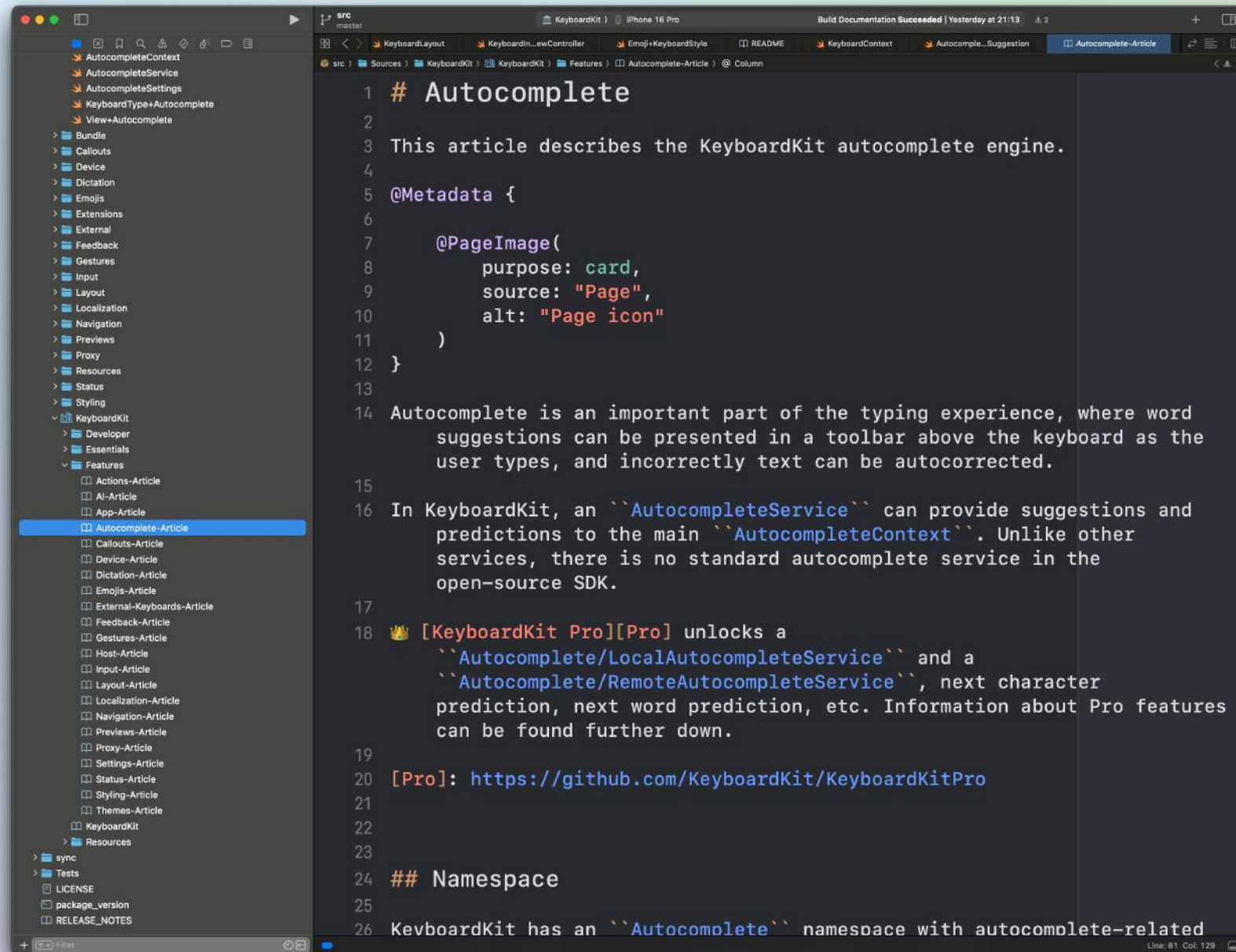


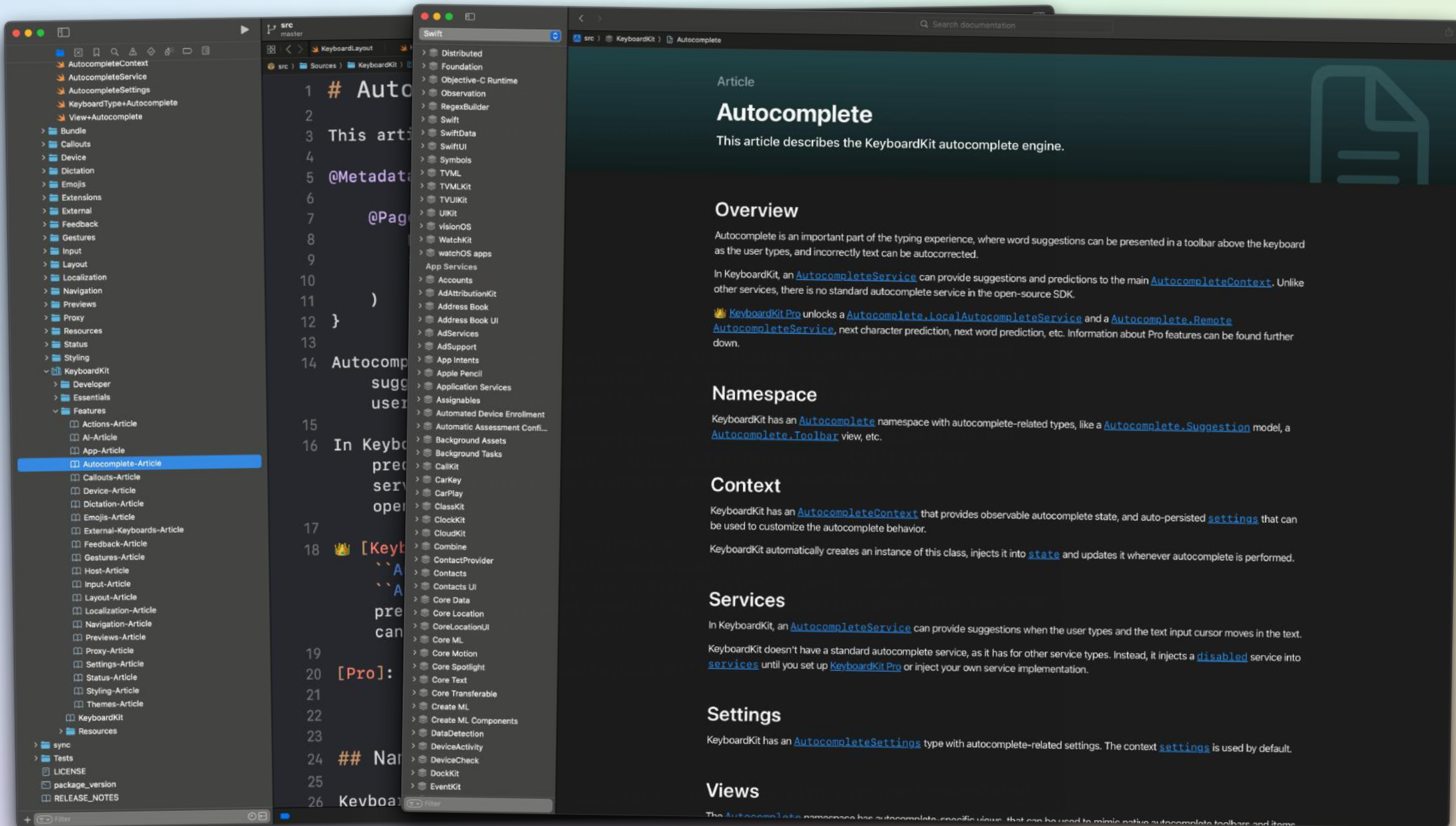
DocC - Code Documentation



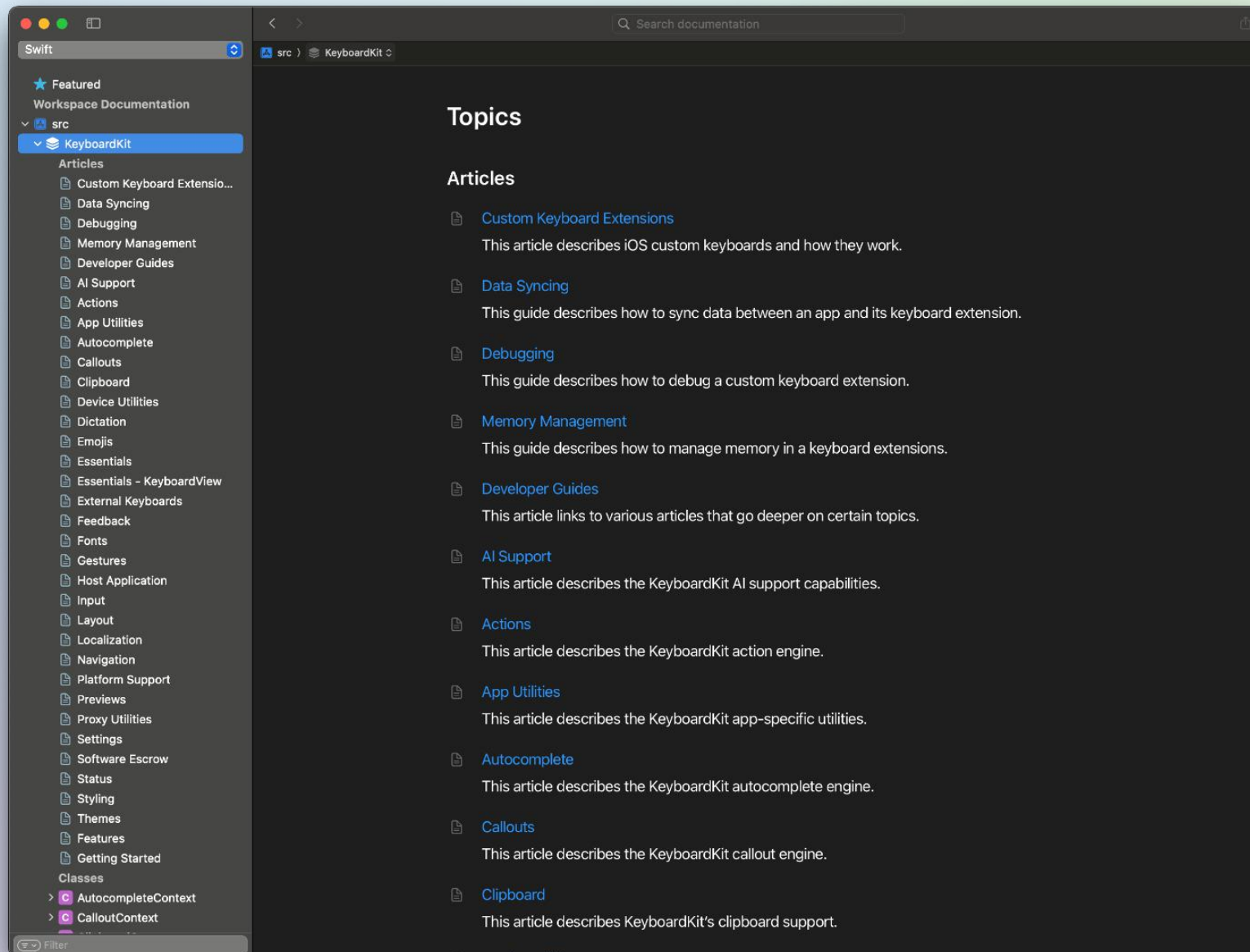


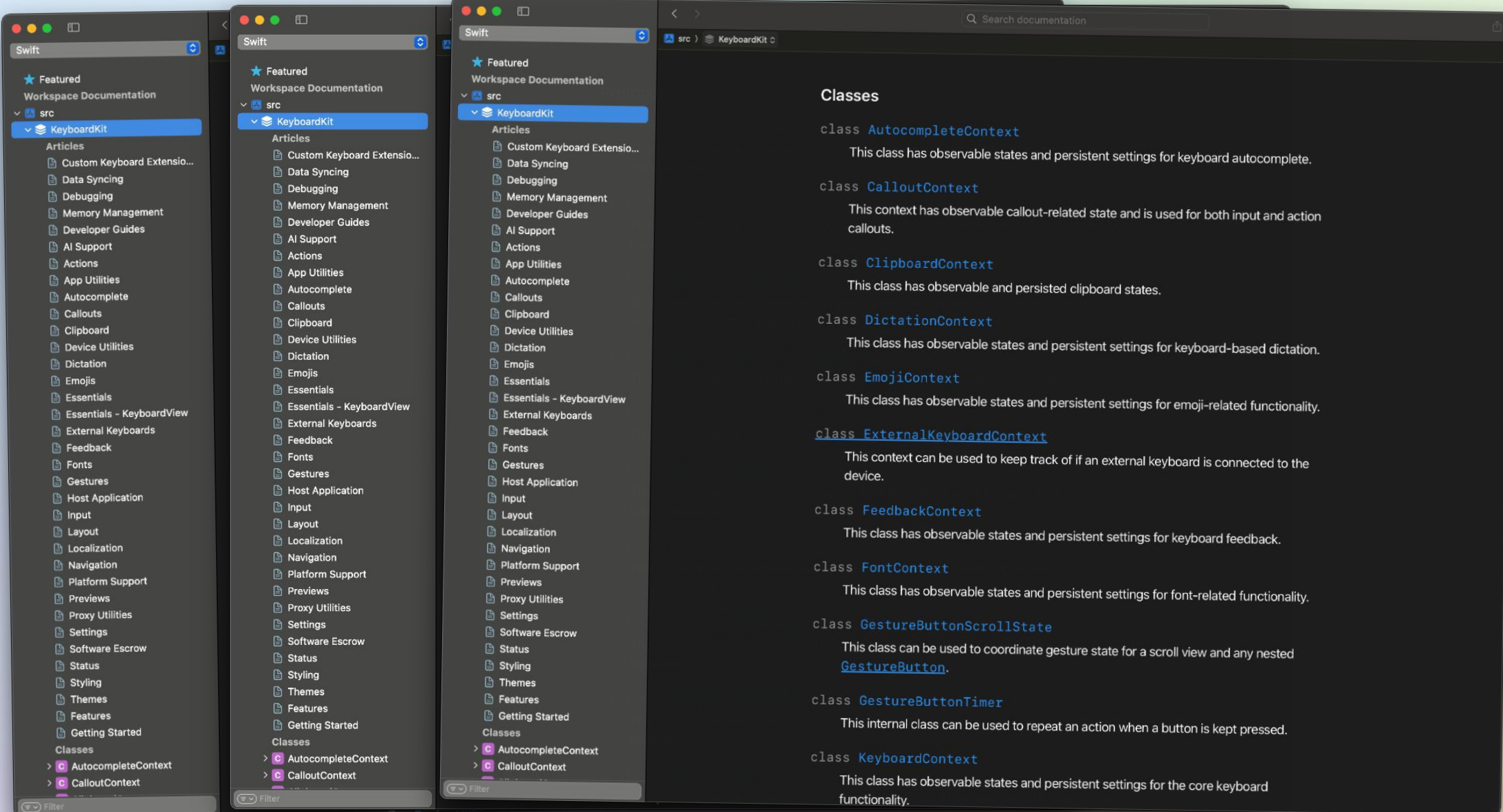
DocC - *Articles*

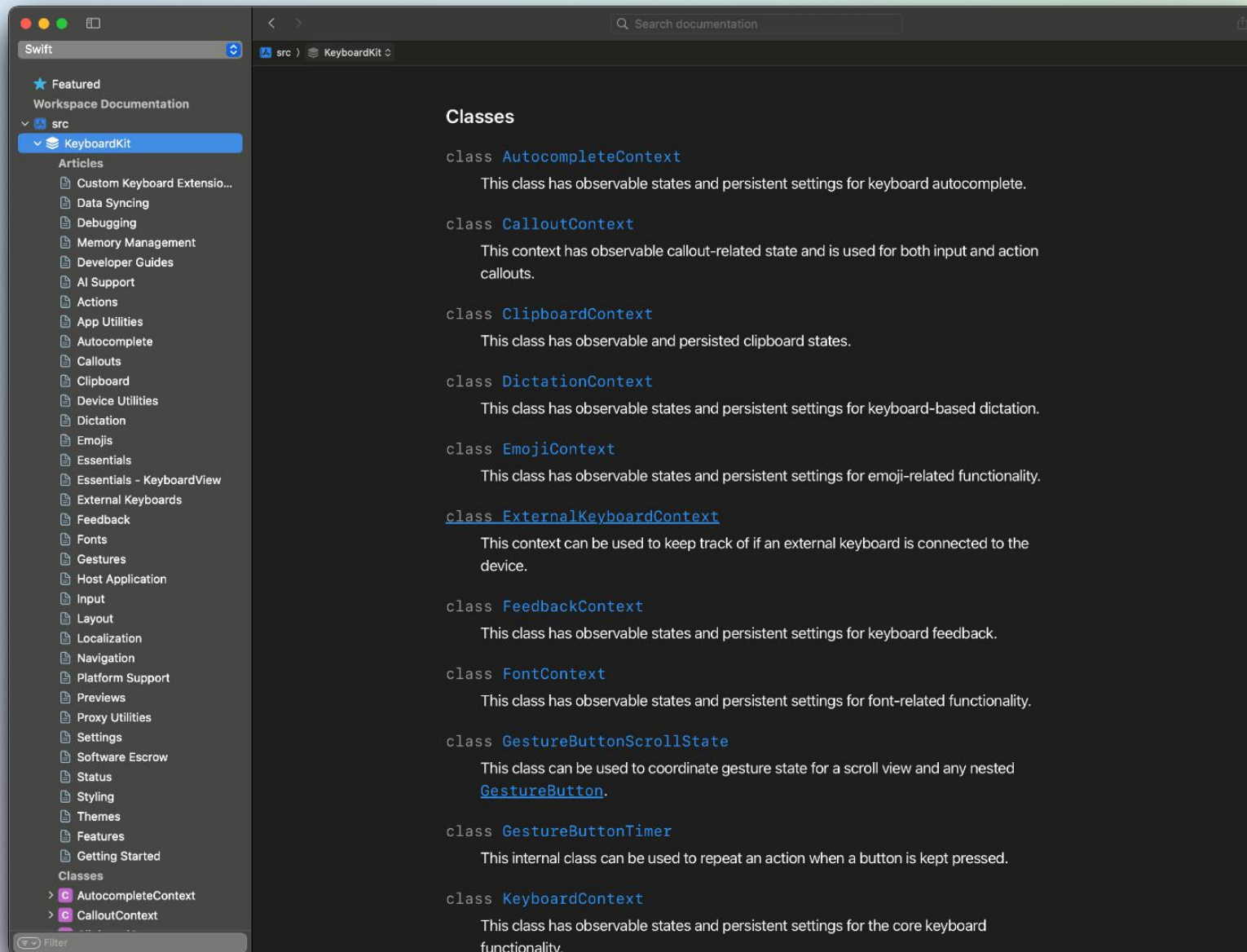




DocC - Topics







Swift

★ Featured

Workspace Documentation

src

KeyboardKit

Articles

Custom Keyboard Extensio...

Data Syncing

Debugging

Memory Management

Developer Guides

AI Support

Actions

App Utilities

Autocomplete

Callouts

Clipboard

Device Utilities

Dictation

Emojis

Essentials

Essentials - KeyboardView

External Keyboards

Feedback

Fonts

Gestures

Host Application

Input

Layout

Localization

Navigation

Platform Support

Previews

Proxy Utilities

Settings

Software Escrow

Status

Styling

Themes

Features

Getting Started

Classes

> AutocompleteContext

> CalloutContext

Filter

< >

Search documentation

src > KeyboardKit

Classes

class AutocompleteContext

This class has observable states and persistent settings for keyboard autocomplete.

class CalloutContext

This context has observable callout-related state and is used for both input and action callouts.

class ClipboardContext

This class has observable and persisted clipboard states.

class DictationContext

This class has observable states and persistent settings for keyboard-based dictation.

class EmojiContext

This class has observable states and persistent settings for emoji-related functionality.

class ExternalKeyboardContext

This context can be used to keep track of if an external keyboard is connected to the device.

class FeedbackContext

This class has observable states and persistent settings for keyboard feedback.

class FontContext

This class has observable states and persistent settings for font-related functionality.

class GestureButtonScrollState

This class can be used to coordinate gesture state for a scroll view and any nested [GestureButton](#).

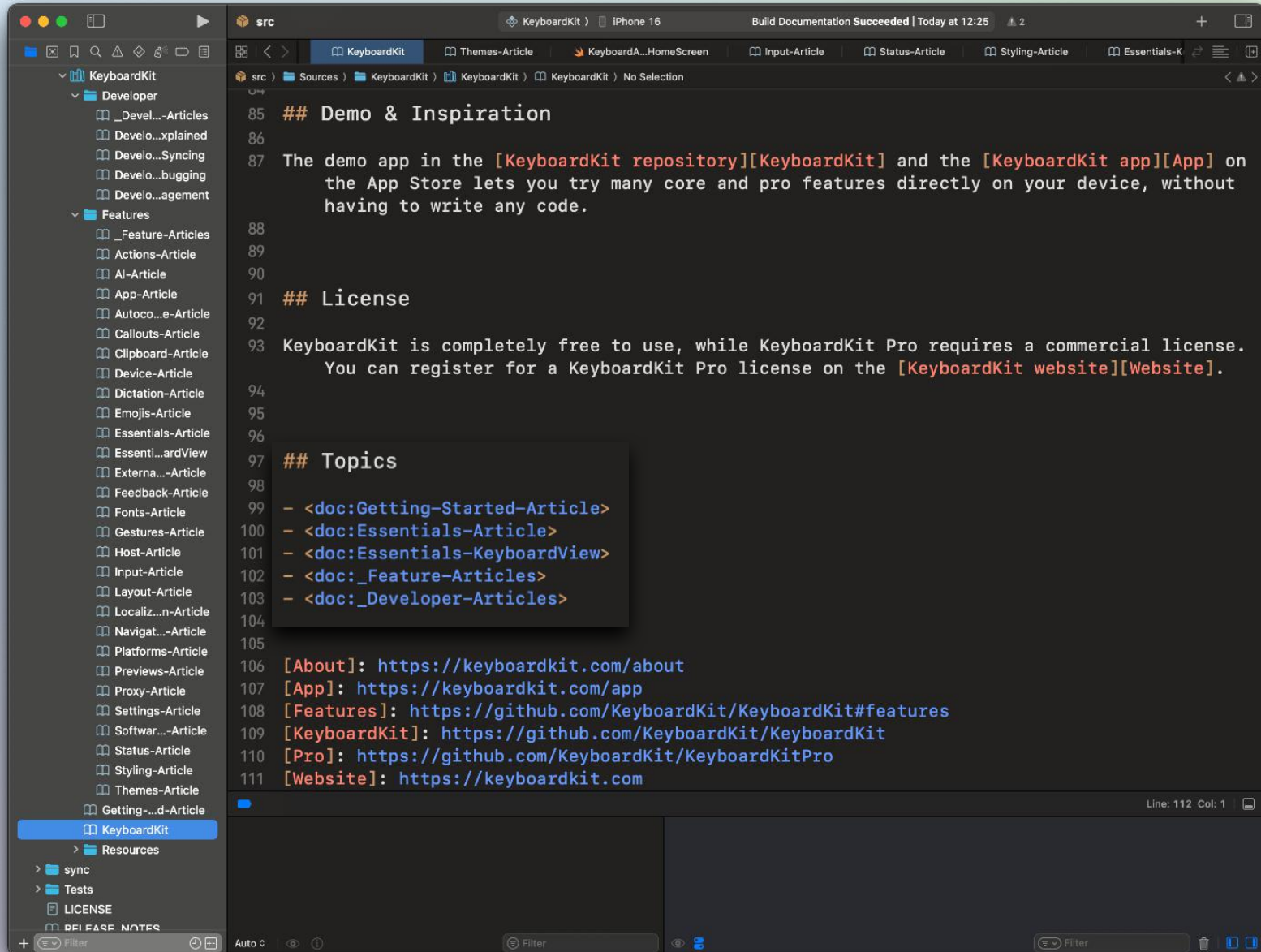
class GestureButtonTimer

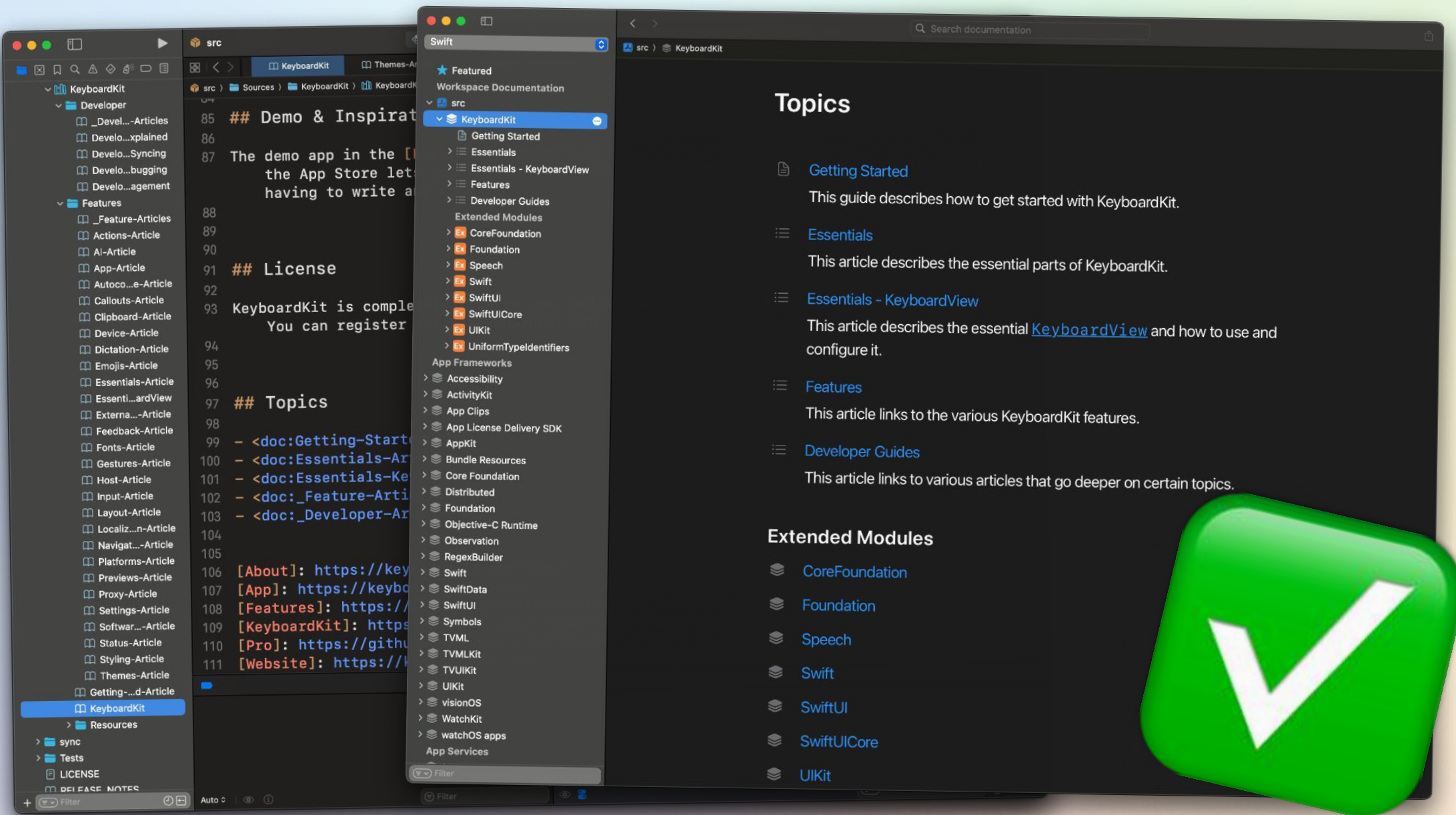
This internal class can be used to repeat an action when a button is kept pressed.

class KeyboardContext

This class has observable states and persistent settings for the core keyboard functionality.







DocC is a great documentation tool!

Use it to document code, create articles, etc.

Versioning

Versioning

Versioning Strategy

Breaking Changes

Semantic Versioning

Deprecations

Breaking Changes

Breaking Changes

Bumping **deployment targets** • Removing or renaming **types** • Removing or renaming **members** • Changing method **signatures** • Adding **async/throwing** • Adding **parameters** without defaults • Removing default parameter **values** • Changing **protocol design** • Adding **enum cases** • Changing class **inheritance** • Making assertions **stricter** • Changing **optionality** • Changing **serialization formats** • etc. etc. etc. etc. **etc. etc.!**

Semantic Versioning

Semantic Versioning

1.X.X

Major Version

Breaking Changes

x.1.x

Minor Version

Big Changes
New Features

x.x.1

Patch Version

Small Changes
Fixes & Tweaks

Semantic Versioning

1.X.X

Major Version

Breaking Changes



x.1.x

Minor Version

Big Changes
New Features

x.x.1

Patch Version

Small Changes
Fixes & Tweaks

Semantic Versioning

1.X.X

Major Version

Breaking Changes



x.1.x

Minor Version

Big Changes
New Features



x.x.1

Patch Version

Small Changes
Fixes & Tweaks

Semantic Versioning

1.X.X

Major Version

Breaking Changes



x.1.x

Minor Version

Big Changes
New Features



x.x.1

Patch Version

Small Changes
Fixes & Tweaks



Semantic version is a contract to your users!
A promise to only break things in major versions.

Deprecations

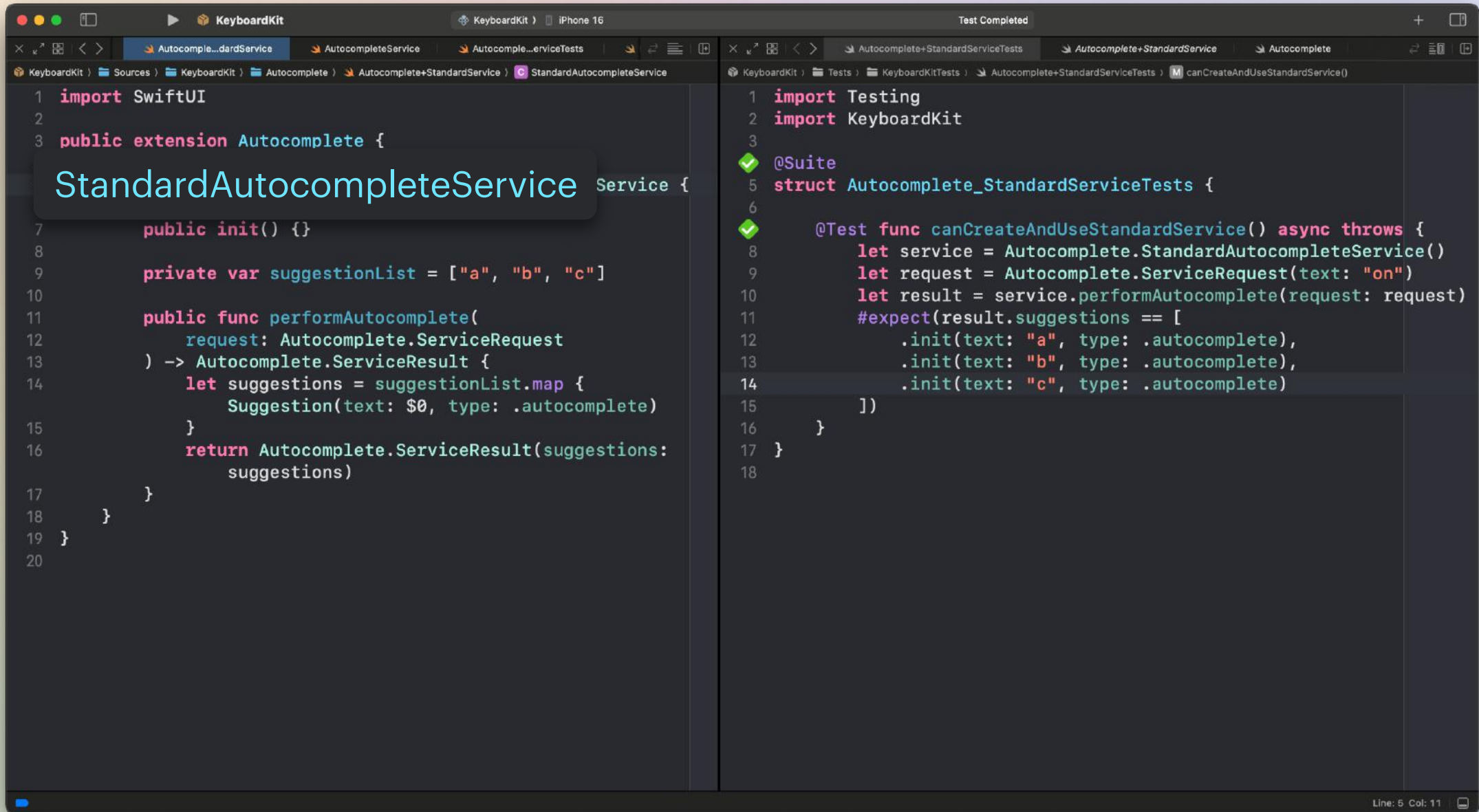
Deprecations

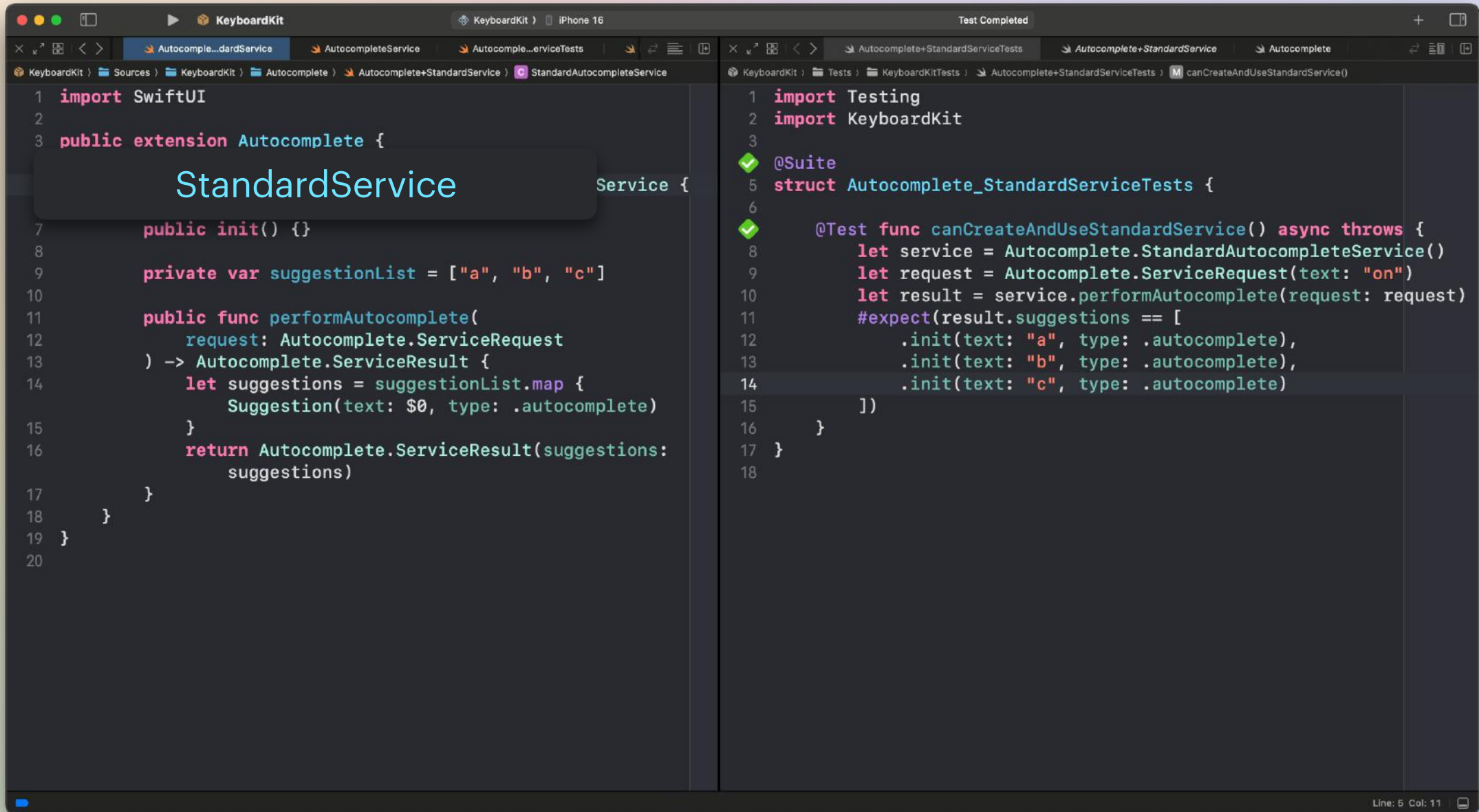
Deprecate what you want to remove or change

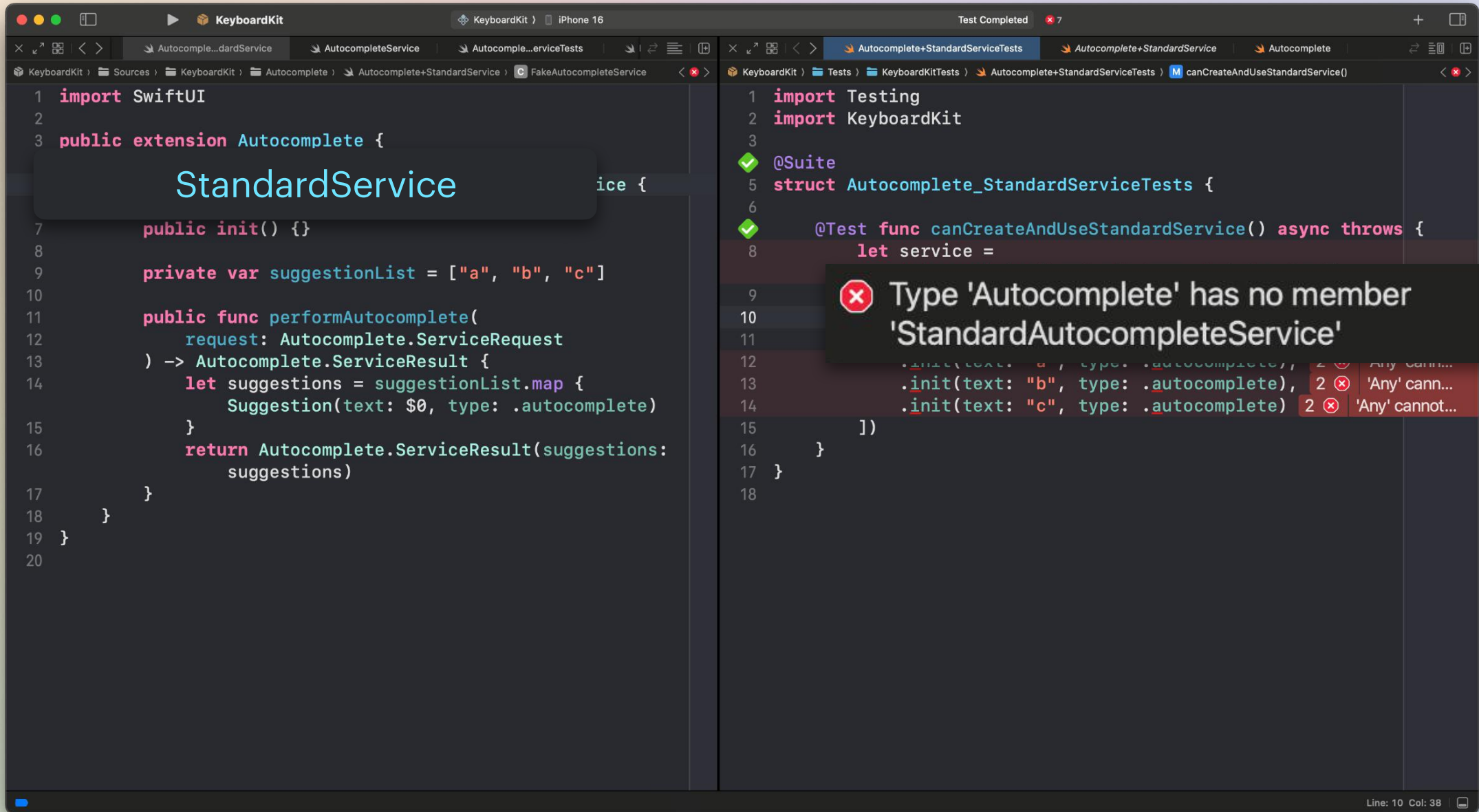
Provide **instructions** for how to adapt the upcoming change

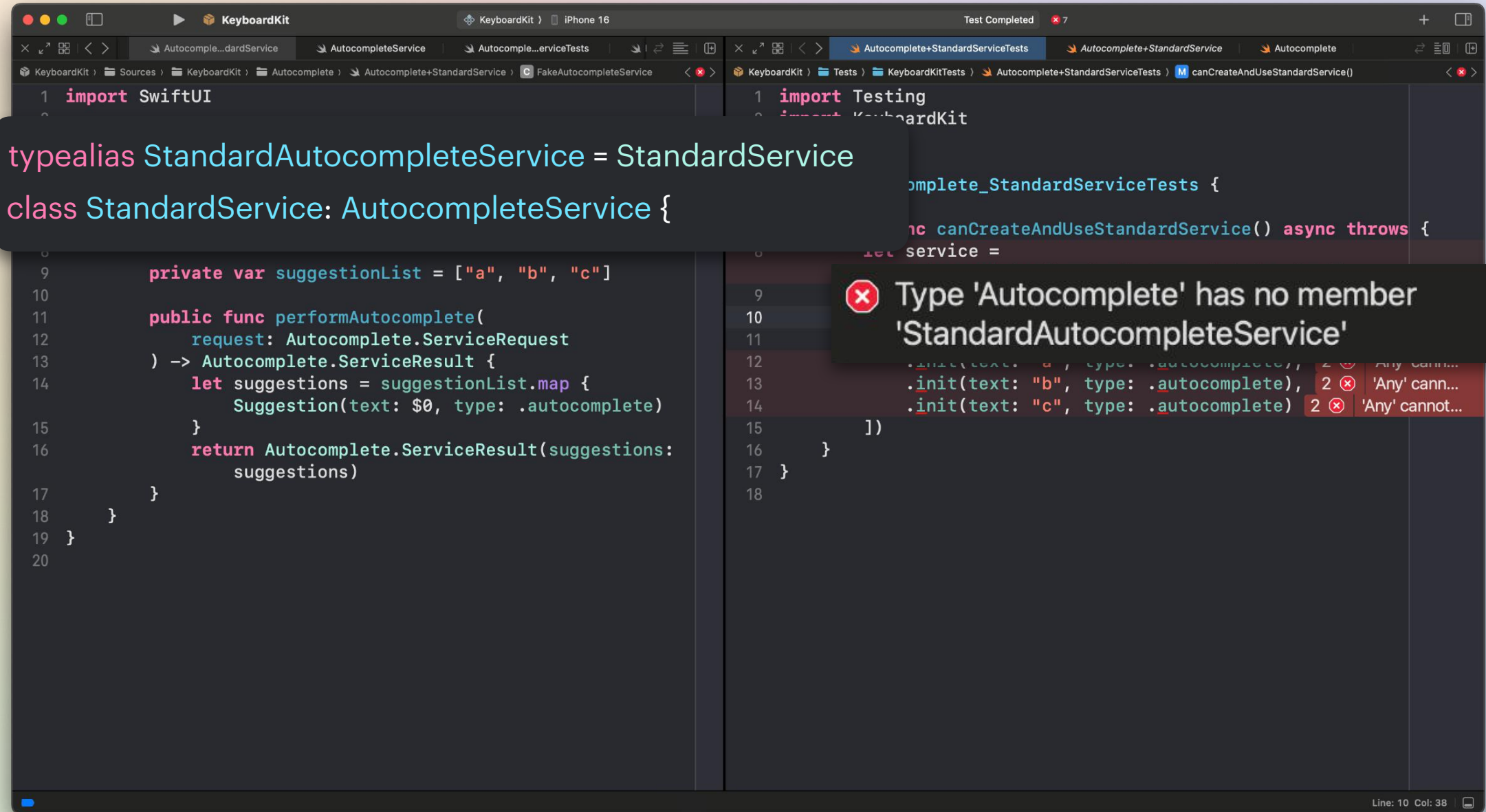
Archive the deprecated code in a **fully working state**

Remove all deprecations in the **next major version** (or later)









`typealias StandardAutocompleteService = StandardService`
`class StandardService: AutocompleteService {`

```
1 import SwiftUI
2
3
4
5
6
7
8     private var suggestionList = ["a", "b", "c"]
9
10    public func performAutocomplete(
11        request: Autocomplete.ServiceRequest
12    ) -> Autocomplete.ServiceResult {
13        let suggestions = suggestionList.map {
14            Suggestion(text: $0, type: .autocomplete)
15        }
16        return Autocomplete.ServiceResult(suggestions:
17            suggestions)
18    }
19 }
20
```

Test Completed

```
1 import Testing
2 import KeyboardKit
3
4 complete_StandardServiceTests {
5
6     func canCreateAndUseStandardService() async throws {
7         let service = Autocomplete.StandardAutocompleteService()
8         let request = Autocomplete.ServiceRequest(text: "on")
9         let result = service.performAutocomplete(request: request)
10        #expect(result.suggestions == [
11            .init(text: "a", type: .autocomplete),
12            .init(text: "b", type: .autocomplete),
13            .init(text: "c", type: .autocomplete)
14        ])
15    }
16 }
17
18
```

Line: 5 Col: 11

@available(*, deprecated, renamed: "StandardService")

typealias StandardAutocompleteService = StandardService

class StandardService: Autocon

```
@Test func canCreateAndUseStandardService() async throws {  
    let service = Autocomplete.StandardAutocompleteService()  
    let re  
    let re  
    #expect  
    .i
```

⚠ 'StandardAutocompleteService' is deprecated: renamed to 'StandardService'

🔧 Use 'StandardService' instead

Fix

```
10  
11  
12     public func performAutocomplete  
13         request: Autocomplete.Serv  
14     ) -> Autocomplete.ServiceResul  
15         let suggestions = suggesti  
16         Suggestion(text: $0, t  
17     }  
18     return Autocomplete.ServiceResult(suggestions:  
19         suggestions)  
20 }
```

```
16 }  
17 }  
18 }
```

`@available(*, deprecated, message: "This class will be removed in 10.0!")`

`typealias StandardAutocompleteService = StandardService`

`class StandardService: AutocompleteService {`
 `@Test func canCreateAndUseStandardService() async throws {`
 `let service = Autocomplete.StandardAutocompleteService()`

⚠️ 'StandardAutocompleteService' is deprecated: This class will be removed in 10.0!

```
10     private var suggestionsList = List<Suggestion>()
11
12     public func performAutocomplete(
13         request: Autocomplete.ServiceRequest
14     ) -> Autocomplete.ServiceResult {
15         let suggestions = suggestionsList
16         return Autocomplete.ServiceResult(suggestions:
17             suggestions)
18     }
19 }
20
```

```
ServiceTests {
16 }
17 }
18
```

Establish & communicate a versioning strategy!

Use deprecations to manage breaking changes.

Conclusion





Thank you!



@danielsaidi

Questions?

